

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение
высшего образования «**Национальный исследовательский
Нижегородский государственный университет им. Н. И.
Лобачевского**»
(ННГУ)

**Институт информационных технологий, математики и
механики**
**Кафедра математического обеспечения и суперкомпьютерных
технологий**

Направление подготовки: «Программная инженерия»

Магистерская диссертация

на тему:

**«ПАРАЛЛЕЛЬНАЯ РЕАЛИЗАЦИЯ
QR-РАЗЛОЖЕНИЯ»**

Выполнил:

студент группы Коробейникова А.Д.

ФИО
(Подпись)

Научный руководитель:

д.т.н., доцент Баркалов К.А.

ФИО
(Подпись)

Нижний Новгород

2026 г.

Содержание (оглавление)

Введение.....	4
Постановка задачи.....	6
1 Теоретическая часть.....	7
1.1 QR-разложение: общие теоретические сведения.....	7
1.2 Описание алгоритмов.....	8
1.2.1 Описание входных и выходных данных.....	8
1.2.3 Метод QR-разложения на основе отражений Хаусхолдера.....	9
1.2.4 Метод QR-разложения на основе вращений Гивенса.....	13
1.3 QR разложение матрицы Хессенберга.....	16
2 Практическая часть.....	19
2.1 Описание разработанной библиотеки QR-decomposition.....	19
2.2 Реализация метода отражений Хаусхолдера.....	23
2.1.2 Первая реализация.....	23
2.1.2 Избавление от матричных умножений.....	27
2.1.3 Работа с нормированными векторами Хаусхолдера.....	32
2.2 Реализация метода вращений Гивенса.....	40
2.2.1 Первая реализация.....	40
2.2.2 Хранение матриц Q и R в одной структуре.....	46
2.2.3 Использование SIMD.....	49
2.2.4 QR-разложение Хессенберговой матрицы.....	56
Заключение.....	60
Список литературы.....	62
Приложение 1. QR-разложение с использованием отражений Хаусхолдера, первая реализация	64
Приложение 2. QR-разложение с использованием отражений Хаусхолдера, реализация без матричных умножений.....	66
Приложение 3. QR-разложение с использованием отражений Хаусхолдера, реализация in-place алгоритма.....	69
Приложение 4. QR-разложение с использованием вращений Гивенса, первая реализация.....	72

Приложение 5. QR-разложение с использованием вращений Гивенса, Q и R в одной структуре данных.....	74
Приложение 6. QR-разложение с использованием вращений Гивенса, векторизованные вычисления.....	76

Введение

Зачастую научно-технические расчёты требуют решения задач линейной алгебры. Моделирование многих физических процессов сводится к системам дифференциальных уравнений, которые в численном решении приводятся к виду систем линейных алгебраических уравнений. Поэтому проблема быстрой обработки таких больших объёмов данных очень актуальна. В этой работе рассматривается алгоритм QR-разложения для представления матрицы в более удобном для быстрых расчётов виде. Кроме того, QR-разложение также является основой одного из методов поиска собственных векторов и чисел матрицы — QR-алгоритма [1]. С помощью этого разложения можно также решить линейную задачу наименьших квадратов и систему линейных алгебраических уравнений.

QR-разложение матрицы – представление матрицы A размера $n \times m$, где $N \geq m$ в виде произведения унитарной (или ортогональной матрицы) Q ($Q^T Q = E$) и верхнетреугольной матрицы R . При $N=m$ матрица Q унитарная. Если при этом A невырождена, то QR-разложение единственно и матрица R может быть выбрана так, чтобы её диагональные элементы были положительными вещественными числами. В частном случае, когда матрица A состоит только из вещественных чисел, матрицы Q и R также могут быть выбраны вещественными, причём Q является ортогональной [1]. На сегодняшний день применяется несколько методов для вычисления QR-разложения матриц [2]. Для ряда методов существуют эффективные программные реализации [3 - 5].

QR-разложение может быть получено различными методами. Проще всего оно может быть вычислено в процессе ортогонализации Грама-Шмидта [6]. Другие методы QR-разложения основаны на отражениях Хаусхолдера и вращениях Гивенса [2, 7]. Все перечисленные методы имеют ряд ограничений для применения [8]:

- Процесс ортогонализации Грама-Шмидта имеет сложность $O(n^3)$; практически не поддаётся распараллеливанию; требует явного хранения Q (n^2 элементов); численно неустойчив, в большинстве случаев приводит к нарушению ортогональности. Модифицированный алгоритм чуть лучше, но и он имеет этот недостаток.
- Отражения Хаусхолдера имеют сложность приблизительно $O(n^3)$; для достижения наибольшей эффективности требуется более сложная блочная реализация; вектора отражений можно хранить под диагональю матрицы R , экономя память; для построения явной Q все равно требуется $O(n^2)$ памяти и дополнительных вычислений; численно устойчив. Является основным методом для разложения плотных матриц

общего вида. Из-за действий над целыми столбцами неэффективен для разреженных матриц, так как не учитывает уже занулённые значения.

- Вращения Гивенса имеют сложность $2n^3$; вектора вращений можно хранить очень компактно или даже не хранить явно; численно устойчив; лучше всего подходит для разреженных и ленточных матриц, так как позволяет обнулять отдельные элементы, не затрагивая другие; в связи с этим медленнее работает для плотных матриц. Этот метод также используется для обновления уже готового разложения при добавлении новой строки или столбца.

В этой работе приводятся параллельные реализации QR-разложения по методам Хаусхолдера и Гивенса с применением технологий параллелизма OpenMP.

Постановка задачи

Необходимо реализовать эффективные программные алгоритмы QR-разложения квадратной матрицы A с помощью методов отражений Хаусхолдера и вращений Гивенса, используя различные аппаратные оптимизации, а также применяя, где это возможно, параллелизм. Разработать несколько версий, сравнить их работу друг с другом.

Для разработки использовать язык Си++ и технологии OpenMP для обеспечения параллельного исполнения кода.

Сгенерировать целочисленную матрицу A размером $N \times N$. Протестировать работу алгоритма на разных размерностях матрицы N : от небольших размеров (100 элементов), до больших объёмов данных ($N > 1000$). Протестировать работу на различном количестве потоков. Сравнить работу алгоритма с известными реализациями в сторонних библиотеках.

Посчитать абсолютную и относительную ошибки для разложения. Критерием правильности считать близость ошибок к какому-либо малому ϵ . Дополнительно проверить зависимость показателей точности и времени от типа данных для представления числа с плавающей точкой: текущий – `float`, одинарной точности, для сравнения — `double`, двойной точности.

В качестве примера приложения QR-разложения на практике выполнить решение задачи QR-разложения хессенберговых матриц, верхней и нижней. Для этого генерировать указанные матрицы и выполнять их разложение методом вращений Гивенса, как наиболее оптимальный способ разложения верхней хессенберговой матрицы (в виду её верхнетреугольной структуры), сравнивая это решение с разложением по методу отражений Хаусхолдера, как наиболее оптимальный способ разложения нижней хессенберговой матрицы (в виду её нижнетреугольной структуры).

Сравнить полученные результаты (время работы разработанной программы, величины абсолютной и относительной ошибок матриц Q и R) с аналогичными результатами, полученными после работы с какой-либо из существующих реализаций из известных библиотек на одних и тех же данных.

1 Теоретическая часть

1.1 QR-разложение: общие теоретические сведения

QR-разложение матрицы – представление матрицы в виде произведения унитарной (или ортогональной матрицы) Q ($Q^T Q = E$) и верхнетреугольной матрицы R с положительными диагональными элементами. QR-разложение является основой одного из методов поиска собственных векторов и чисел матрицы — QR-алгоритма. С помощью разложения можно решить линейную задачу наименьших квадратов.

Также с помощью QR-разложения можно решать системы линейных алгебраических уравнений (СЛАУ). В матричном виде эта задача описывается как (A – матрица системы, x – столбец неизвестных, b – столбец свободных членов). Если известно QR-разложение матрицы, исходная система может быть записана как $QRx = b$. Решением этой задачи является решение треугольной системы $Rx = Q^T b$. Эта система может быть решена с помощью обратной подстановки, поскольку R – верхняя треугольная матрица.

В работе в качестве примера приложения QR-разложения на практике рассматривается решение задачи QR-разложения хессенберговых матриц, верхней и нижней. Матрицы Хессенберга получаются как продукт методов подпространства Крылова в процессе построения ортогональных базисов, а также в задаче на нахождение собственных значений матрицы QR-методом.

QR-разложение может быть получено различными методами. Проще всего оно может быть вычислено в процессе ортогонализации Грама-Шмидта. Другие методы QR-разложения основаны на отражениях Хаусхолдера и вращениях Гивенса.

1.2 Описание алгоритмов

1.2.1 Описание входных и выходных данных

Генерируется квадратная плотная матрица A размером $N \times N$. Значения для A генерируются в интервале $[-10; 10]$.

Полученные в результате работы алгоритма QR-разложения матрицы Q и R необходимо перемножить и сравнить результат с исходной матрицей A .

Подсчитаем абсолютную ошибку полученного QR-разложения с помощью Нормы Фробениуса (также евклидова норма, соответствующая евклидову пространству матриц относительно фробениусова скалярного произведения, или сферическая норма) представляет собой частный случай p -нормы для $p = 2$: $\|A\|_F = \sqrt{\sum_{i=0}^n \sum_{j=0}^m a_{ij}^2}$, где n, m – размеры матрицы A .

Получим $\|QR - A\|_F$ и $\frac{\|QR - A\|_F}{\|A\|_F}$ для оценки ошибки.

Для сравнения результатов по итогам запуска разработанной программы была выбрана реализация QR-разложения из библиотеки Scilab. В Scilab QR-разложение реализуется с помощью функции `qr`. Она позволяет разложить матрицу X на ортогональную, в вещественном случае, матрицу Q и верхнюю треугольную матрицу R , при этом выполняется равенство $X = QR$. QR-разложение в Scilab основано на подпрограммах Lapack: DGEQRF, DGEQPF, DORGQR для вещественных матриц.

1.2.3 Метод QR-разложения на основе отражений Хаусхолдера

Одним из самых распространённых методов нахождения QR-разложения является метод отражений Хаусхолдера. Суть алгоритма QR-разложения, реализованного с помощью отражений Хаусхолдера – в последовательном домножении матрицы A слева на отражения Хаусхолдера H с целью получения верхнетреугольной матрицы R как побочного продукта и Q с помощью явных перемножений в простейшем случае.

Пусть v – N -мерный ненулевой вектор-столбец. Квадратная матрица H порядка N вида:

$$H = E - \frac{2vv^T}{v^Tv}$$

называется отражением Хаусхолдера или просто отражением. Вектор v называется вектором Хаусхолдера. Умножению матрицы H на вектор x можно дать следующую геометрическую интерпретацию: вектор Hx получается отражением вектора x относительно гиперплоскости, ортогональной вектору v . Матрица Хаусхолдера будет симметрична и ортогональна.

Пусть $x \neq 0$. Найдём отражение H , такое, что $Hx = \alpha e_1$ для какого-либо α . Имеем:

$$Hx = \left(E - \frac{2vv^T}{v^Tv} \right) x = x - \frac{2v(v^Tx)}{v^Tv} = x - \frac{2v^Tx}{v^Tv} v$$

и поэтому v коллинеарен $x - Hx = x - \alpha e_1$. Так как v определен с точностью до ненулевого множителя, то можно положить $v = x - \alpha e_1$. Ввиду ортогональности матрицы H имеем $\|Hx\|_2 = \|x\|_2$, откуда $\alpha = \pm \|x\|_2$. Поэтому $v = x \pm \|x\|_2 e_1$.

Итак, в качестве вектора Хаусхолдера v можно взять вектор, вычисляемый по формуле $v = x \pm \|x\|_2 e_1$, или любой ненулевой, ему коллинеарный.

С алгоритмом построения вектора Хаусхолдера связаны некоторые важные детали. Одна из них — выбор знака в формуле $v = x \pm \|x\|_2 e_1$. Если x почти коллинеарен e_1 , то вектор $v = x - \text{sign}(x_1) \|x\|_2 e_1$ имеет малую норму. Вследствие этого возможно появление большой относительной ошибки при вычислении множителя $2/v^T v$. Эту трудность можно обойти, взяв α с тем же знаком, что и первая компонента вектора x : $v = x - \text{sign}(x_1) \|x\|_2 e_1$.

Это обеспечивает выполнение неравенства $\|v\|_2 \geq \|x\|_2$ и гарантирует почти точную ортогональность вычисленной матрицы H .

Другая деталь связана с выбором множителя для вектора, вычисляемого по формуле $v = x - \text{sign}(x_1) \|x\|_2 e_1$. Можно использовать такой множитель, что $v_1 = 1$. Это несколько

упрощает способ хранения ортогональных матриц, вычисленных по методу отражений Хаусхолдера.

На каждом шаге алгоритма k получаем k -ое отражение H_k , домножая на него матрицу A слева получаем $\widetilde{A}_k = H_k H_{k-1} \dots H_1 A$, в которой k первых столбцов имеют нули под главной диагональю:

$$H_1 A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & f_{32} & f_{33} & f_{34} \\ 0 & f_{42} & f_{43} & f_{44} \end{pmatrix}, H_2 H_1 A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & 0 & f_{33} & f_{34} \\ 0 & 0 & f_{43} & f_{44} \end{pmatrix}, H_3 H_2 H_1 A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & 0 & f_{33} & f_{34} \\ 0 & 0 & 0 & f_{44} \end{pmatrix}$$

На каждом шаге алгоритма получается вектор Хаусхолдера v и матрица отражения H :

$$H = E - \frac{v \cdot v^T}{v^T \cdot v}$$

Домножая матрицу A слева на матрицы отражения H_k , получим матрицу R :

$$A_k = H_{k-1} \dots H_1 A,$$

$$R = H_{n-1} \dots H_1 A$$

Матрица Q при этом хранится и также вычисляется на каждом шаге с помощью последовательных домножений справа всех матриц отражения:

$$Q = H_1 \dots H_{n-1}.$$

Выполнив N таких шагов, получим верхнюю треугольную матрицу $R = H_n H_{n-1} \dots H_1 A$. Откуда, так как матрицы H_j – ортогональные и симметричные, имеем $A = QR$, где $Q = H_1 H_2 \dots H_n$.

Самая очевидная реализация метода Хаусхолдера будет содержать в себе следующие недостатки:

- использование операции матричного умножения матриц размером $N \times N$; на каждую из $(N-1)$ итераций цикла придётся два матричных произведения.
- отдельное хранение матриц отражения H ($N \times N$ элементов, $(N-1)$ штук) для вычисления Q .

В связи с выявленными недостатками возможны следующие оптимизации:

1. Избавление от матричных умножений

Подсчитаем перемножение $H_k A_k$ по-другому:

$$H_k A_k = (I - 2 v \cdot v^T) A_k = A_k - 2 v (v^T \cdot A_k),$$

получим более дешёвые операции над векторами и матрицей A , уменьшим сложность в N раз.

2. Также вычисление матрицы R этим способом позволяет упростить вычисление матрицы Q . Для её восстановления понадобится только сохранить вектора Хаусхолдера v_k , полученные на каждом шаге алгоритма. Вычислять Q будем восстанавливая матрицы отражения из сохранённых векторов v , в обратном порядке:

$$\begin{aligned} Q_{n-1} &= I, \\ Q_k &= H_k Q_{k-1} = Q_{k-1} - 2v_k(v_k^T Q_{k-1}), k = n-2, \dots, 0, \\ Q &= Q_0 \end{aligned}$$

Получим экономию памяти порядка N и ускорение времени больше, чем на порядок.

3. Работа с нормированными векторами Хаусхолдера позволит формировать вектор v_k , с $v_{kk}=1$:

$$\begin{aligned} \sigma &= -\text{sign}(a_k[0]) \|a_k\|, \\ v_k &= a_k - \sigma e_k = \begin{bmatrix} 0; \dots; 0; \underbrace{a_{kk} - \sigma}_{k-1}; a_{k,k+1}; \dots; a_{k,n} \end{bmatrix}, \\ H &= I - \tau * (v \cdot v^T), \text{ где } \tau = \frac{2}{v^T \cdot v}, \\ H_k A_k &= (I - \tau v \cdot v^T) A_k = A_k - \tau v (v^T A_k) \end{aligned}$$

4. Если изменить способ вычисления вектора Хаусхолдера v_k , на тот, что указан в п.3, вычисляя нормированный вектор с незначащими $(k-1)$ первыми позициями и $(k+1)$ элементом, равным единице, то матрицы H_k не потребуется формировать и сохранять в явном виде: достаточно хранить соответствующий вектор v_k , который можно записывать в k -й столбец матрицы A под диагональю вместо незначащих нулей следующим образом:

$$H_1 A = A_1 = \begin{bmatrix} \sigma_1 & \widetilde{A_{12}} & \dots & \widetilde{A_{1n}} \\ v_{11}/v_{10} & \widetilde{A_{22}} & \dots & A_{2n} \\ \dots & \dots & \dots & \dots \\ v_{1k}/v_{10} & \widetilde{A_{k2}} & \dots & \widetilde{A_{kn}} \\ v_{1,k+1}/v_{10} & \widetilde{A_{k+1,2}} & \dots & \widetilde{A_{k+1,n}} \\ \dots & \dots & \dots & \dots \\ v_{1n}/v_{10} & \widetilde{A_{n2}} & \dots & \widetilde{A_{n,n}} \end{bmatrix},$$

$$H_2 A_1 = A_2 = \begin{bmatrix} \sigma_1 & \widetilde{A}_{12} & \cdots & \widetilde{A}_{1n} \\ v_{11}/v_{10} & \sigma_2 & \cdots & A_{2n} \\ v_{12}/v_{10} & v_{21}/v_{20} & \cdots & A_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ v_{1k}/v_{10} & v_{2k}/v_{20} & \cdots & \widetilde{A}_{kn} \\ v_{1,k+1}/v_{10} & v_{2,k+1}/v_{20} & \cdots & \widetilde{A}_{k+1,n} \\ \cdots & \cdots & \cdots & \cdots \\ v_{1n}/v_{10} & v_{2,n}/v_{20} & \cdots & \widetilde{A}_{n,n} \end{bmatrix},$$

$$H_k A_{k-1} = \begin{bmatrix} \sigma_1 & \widetilde{A}_{12} & \cdots & \widetilde{A}_{1k} & \cdots & \cdots & \widetilde{A}_{1n} \\ v_{11}/v_{10} & \sigma_2 & \cdots & \widetilde{A}_{2k} & \cdots & \cdots & A_{2n} \\ \cdots & \cdots & \ddots & \cdots & \cdots & \cdots & \cdots \\ v_{1k}/v_{10} & \cdots & \cdots & \sigma_k & \cdots & \cdots & \widetilde{A}_{kn} \\ v_{1,k+1}/v_{10} & \cdots & \cdots & v_{k1}/v_{k0} & \widetilde{A}_{k+1,k+1} & \cdots & \widetilde{A}_{k+1,n} \\ \cdots & \cdots & \cdots & \vdots & \vdots & \ddots & \cdots \\ v_{1n}/v_{10} & \cdots & \cdots & v_{kn}/v_{k0} & \widetilde{A}_{n,k+1} & \cdots & \widetilde{A}_{n,n} \end{bmatrix}$$

При этом отражение заключается в применении к столбцам справа от обнуляемого, от (k+1) до (N-1), преобразования $R - \tau v(v^T R)$, где $(v^T R)$ — скалярное произведение вектора Хаусхолдера из столбца col, вычитание из R выполняется над тем же столбцом.

Отдельно вычисляется матрица Q с помощью сохранённых под диагоналями нормированных векторов Хаусхолдера, как в предыдущей реализации, только вместо 2 используется $\tau = \frac{2}{v^T v}$ в качестве коэффициента. Работа также идёт только над столбцами, от k до N: скалярное умножение с вектором и вычитание вектора из столбца.

1.2.4 Метод QR-разложения на основе вращений Гивенса

Метод Гивенса также выполняет разложение матрицы A на Q и R , но с помощью операции вращения. При этом матрица Q хранится и используется не в явном виде, а в виде произведения матриц вращения. Каждая из матриц вращения (Гивенса) представлена позициями столбцов i и j и параметрами $\cos$ и $\sin$ вращения:

$$G_{ij} = \begin{bmatrix} 1 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \dots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & 0 \\ 0 & \dots & 1 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & c & 0 & \dots & 0 & -s & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & s & 0 & \dots & 0 & c & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Суть алгоритма QR-разложения, реализованного с помощью вращений Гивенса – в последовательном занулении всех поддиагональных элементов, с целью получения верхнетреугольной матрицы R как побочного продукта и Q с помощью явных перемножений в простейшем случае. На каждом шаге алгоритма две строки i и j преобразуемой матрицы подвергаются преобразованию вращения. При этом параметр такого преобразования подбирается так, чтобы один из поддиагональных элементов преобразуемой матрицы стал нулевым.

На каждом шаге алгоритма для зануления элемента с индексами (i,j) получаем вращение G_{ji} , домножая на него матрицу A слева получаем $\widetilde{A}_k = G_{ji} G_{j,(i-1)} \dots G_{12} A$, в которой элементы под главной диагональю, левее (i,j) , занулены:

$$G_{12} A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ f_{13} & f_{32} & f_{33} & f_{34} \\ f_{14} & f_{42} & f_{43} & f_{44} \end{pmatrix}, G_{13} G_{12} A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & f_{32} & f_{33} & f_{34} \\ f_{14} & f_{42} & f_{43} & f_{44} \end{pmatrix}, G_{14} G_{13} G_{12} A = \begin{pmatrix} f_{11} & f_{12} & f_{13} & f_{14} \\ 0 & f_{22} & f_{23} & f_{24} \\ 0 & f_{32} & f_{33} & f_{34} \\ 0 & f_{42} & f_{43} & f_{44} \end{pmatrix}$$

Сначала в нуль последовательно обращаются элементы 1-го столбца, затем 2-го, и т.д. до $N-1$ -го, после чего получившаяся матрица – это и есть матрица R .

Сам шаг алгоритма распадается на две части: подбор параметра вращения и выполнение вращения над двумя строками матрицы.

Вращение элементов строк в текущем столбце выполняется одновременно с подбором параметра вращения. Таким образом, вторая часть шага заключается в выполнении вращения

двумерных векторов, составленных из элементов вращаемых строк в столбцах справа от "рабочего".

Для получения матрицы R матрицу A последовательно домножают слева на матрицы вращения $G_{12}, G_{13}, \dots, G_{1n}, G_{23}, G_{24}, \dots, G_{2n}, \dots, G_{N-2,N}, G_{N-1,N}$. Каждая из матриц G задаёт вращение в двухмерном подпространстве, связанном с i-й и j-й компонентами столбцов, остальные компоненты не меняются. При этом вращение подбирается так, чтобы элемент новой матрицы в j-й строке и i-м столбце стал нулевым. Поскольку вращения и тождественные преобразования нулевых векторов оставляют их нулевыми, то последующие вращения сохраняют полученные ранее нули слева и сверху от текущего обнуляемого элемента.

$$R = G_{n-1,n} G_{n-2,n} \dots G_{2n} G_{1n} \dots G_{24} G_{23} \dots G_{13} G_{12} A.$$

Отсюда для получения матрицы Q необходимо последовательно перемножить все транспонированные матрицы вращения друг на друга справа.

$$Q = (G_{n-1,n} G_{n-2,n} \dots G_{2n} G_{1n} \dots G_{24} G_{23} \dots G_{13} G_{12})^T = G_{12}^T G_{13}^T \dots G_{1n}^T G_{23}^T G_{24}^T \dots G_{2n}^T \dots G_{n-2,n}^T G_{n-1,n}^T$$

Выполнив $\left(\frac{N^2 - N}{2}\right)$ таких шагов, получим верхнюю треугольную матрицу R и унитарную матрицу Q.

Самая очевидная реализация метода Гивенса будет содержать в себе следующие недостатки:

- неэффективное, «прямое» хранение матрицы Q: обращение к элементам матрицы Q происходит по столбцам, в виду чего выгоднее хранить её транспонированной.
- вычислять R и Q можно в рамках одной итерации цикла, после вычисления параметров вращения, а можно в разных циклах, сохраняя параметры вращения в отдельной структуре либо в матрице R. Необходимо сравнение двух подходов.
- можно не выполнять вращение строки при вычислении матрицы R целиком, поскольку слева от текущего столбца элементы вращаемых строк равны 0.
- выигрыш по ускорению может дать совместное попарное хранение элементов Q и R благодаря оптимизации доступа к памяти. При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи. Это и совместное обновление матриц в одном цикле может дать хорошее ускорение для этой реализации.

- можно ещё эффективнее обращаться к памяти, выполняя вычисления сразу над несколькими последовательно лежащими элементами, применив так называемую векторизацию.

1.3 QR разложение матрицы Хессенберга

Матрица Хессенберга — вид квадратной матрицы, обобщающий треугольные матрицы. Названа так в честь немецкого математика, Карла Хессенберга.

Верхняя матрица Хессенберга — это квадратная матрица, все элементы ниже побочной диагонали которой равны нулю:

$$H_{upper} = \begin{pmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2n} \\ 0 & h_{32} & h_{33} & \cdots & h_{3n} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{nn-1} & h_{nn} \end{pmatrix}$$

Аналогично определяется нижняя матрица Хессенберга, как квадратная матрица, при транспонировании которой получается верхняя матрица Хессенберга:

$$H_{low} = \begin{pmatrix} h_{11} & h_{12} & 0 & \cdots & 0 \\ h_{21} & h_{22} & h_{23} & \cdots & \vdots \\ h_{31} & h_{32} & h_{33} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & h_{n-1n} \\ h_{n1} & h_{n2} & h_{n3} & \cdots & h_{nn} \end{pmatrix}$$

Таким образом, матрица, являющаяся одновременно и верхней, и нижней матрицами Хессенберга, как раз и будет трёхдиагональна.

$$H_{low\ and\ upper} = \begin{pmatrix} a_{11} & a_{12} & 0 & \cdots & \cdots & 0 \\ a_{21} & a_{22} & a_{23} & 0 & \cdots & \vdots \\ 0 & a_{32} & a_{33} & a_{34} & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & \cdots & 0 & a_{nn-1} & a_{nn} \end{pmatrix}$$

Пусть верхняя хессенбергова матрица А имеет размер 6х6:

$$H_{upper} = \begin{pmatrix} x & x & x & x & x & x \\ x & x & x & x & x & x \\ 0 & x & x & x & x & x \\ 0 & 0 & x & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix}$$

С помощью алгоритма QR-разложения на основе вращений Гивенса решение задачи разложения верхней Хессенберговой матрицы выглядит следующим образом. Необходимо с

помощью вращений Гивенса последовательно занулить (N-1) элементов, лежащих на побочной диагонали под главной (выделены красным):

$$H_{upper} = A = \begin{pmatrix} x & x & x & x & x & x \\ \textcolor{red}{x} & x & x & x & x & x \\ 0 & \textcolor{red}{x} & x & x & x & x \\ 0 & 0 & \textcolor{red}{x} & x & x & x \\ 0 & 0 & 0 & \textcolor{red}{x} & x & x \\ 0 & 0 & 0 & 0 & \textcolor{red}{x} & x \end{pmatrix}$$

После первых двух шагов алгоритма QR-разложения с использованием вращений Гивенса обнулены элементы с индексами 1 и 2, а также 2 и 3 (выделены красным цветом), следующий шаг должен обнулить элемент с индексами 4 и 3 (выделен синим цветом) с помощью вращения G_{34} :

$$G_{23} \cdot G_{12} \cdot A = \begin{pmatrix} x & x & x & x & x & x \\ \textcolor{red}{0} & x & x & x & x & x \\ 0 & \textcolor{red}{0} & x & x & x & x \\ 0 & 0 & \textcolor{blue}{x} & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix}$$

Применим вращение G_{34} и получим:

$$G_{34} \cdot G_{23} \cdot G_{12} \cdot A = \begin{pmatrix} x & x & x & x & x & x \\ \textcolor{red}{0} & x & x & x & x & x \\ 0 & \textcolor{red}{0} & x & x & x & x \\ 0 & 0 & \textcolor{red}{0} & x & x & x \\ 0 & 0 & 0 & x & x & x \\ 0 & 0 & 0 & 0 & x & x \end{pmatrix}$$

Таким образом, обобщая, для верхней хессенберговой матрицы алгоритм QR-разложения будет следующим:

$$R = G_{n-1} \cdot \dots \cdot G_1 \cdot A, \text{ где } G_j = G_{j,j+1}, j \in [1; N-1]$$

$$Q^T \cdot A = R,$$

$$Q = G_1^T \cdot \dots \cdot G_{n-1}^T.$$

Пусть нижняя хессенбергова матрица A имеет размер 6x6:

$$H_{lower} = \begin{pmatrix} x & x & 0 & 0 & 0 & 0 \\ x & x & x & 0 & 0 & 0 \\ x & x & x & x & 0 & 0 \\ x & x & x & x & x & 0 \\ x & x & x & x & x & x \\ x & x & x & x & x & x \end{pmatrix}$$

С помощью алгоритма QR-разложения на основе отражений Хаусхолдера решение задачи разложения нижней Хессенберговой матрицы выглядит следующим образом. Необходимо с помощью отражений Хаусхолдера последовательно занулить $\left(\frac{N^2 - N}{2}\right)$ элементов, лежащих ниже главной диагонали (выделены красным):

$$H_{low} = A = \begin{pmatrix} x & x & 0 & 0 & 0 & 0 \\ \textcolor{red}{x} & x & x & 0 & 0 & 0 \\ \textcolor{red}{x} & \textcolor{red}{x} & x & x & 0 & 0 \\ \textcolor{red}{x} & \textcolor{red}{x} & \textcolor{red}{x} & x & x & 0 \\ \textcolor{red}{x} & \textcolor{red}{x} & \textcolor{red}{x} & \textcolor{red}{x} & x & x \\ \textcolor{red}{x} & \textcolor{red}{x} & \textcolor{red}{x} & \textcolor{red}{x} & \textcolor{red}{x} & x \end{pmatrix}$$

После первых двух отражений H_2 и H_1 получим нули под главной диагональю в первых двух столбцах (выделены красным), Следующий шаг должен обнулить элементы столбца 3 с помощью отражения H_3 (выделен синим):

$$H_2 \cdot H_1 \cdot A = \begin{pmatrix} x & x & 0 & 0 & 0 & 0 \\ \textcolor{red}{0} & x & x & 0 & 0 & 0 \\ \textcolor{red}{0} & \textcolor{red}{0} & x & x & 0 & 0 \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{blue}{x} & x & x & 0 \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{blue}{x} & x & x & x \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{blue}{x} & x & x & x \end{pmatrix}$$

Применим отражение H_3 для обнуления следующего столбца и получим:

$$H_3 \cdot H_2 \cdot H_1 \cdot A = \begin{pmatrix} x & x & x & 0 & 0 & 0 \\ \textcolor{red}{0} & x & x & 0 & 0 & 0 \\ \textcolor{red}{0} & \textcolor{red}{0} & x & x & 0 & 0 \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{red}{0} & x & x & 0 \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{red}{0} & x & x & x \\ \textcolor{red}{0} & \textcolor{red}{0} & \textcolor{red}{0} & x & x & x \end{pmatrix}$$

Таким образом, обобщая, для нижней хессенберговой матрицы алгоритм QR-разложения не будет отличаться от разложения произвольной плотной квадратной матрицы и будет следующим:

$$R = H_{n-1} \cdot \dots \cdot H_1 \cdot A$$

$$Q^T \cdot A = R,$$

$$Q = H_1^T \cdot \dots \cdot H_{n-1}^T.$$

2 Практическая часть

2.1 Описание разработанной библиотеки QR-decomposition

Класс для вызова QR-разложения для матрицы описан в интерфейсе IQRSolver. Он содержит единственный абстрактный метод `QR_decomposition` для выполнения разложения, который будет определён в классе с конкретным методом. Сигнатура функции QR-разложения внутри этого интерфейса:

```
void QR_decomposition(const std::vector<std::vector<T>>& A,  
std::vector<std::vector<T>>& Q, std::vector<std::vector<T>>& R)
```

Функция принимает ссылку на неизменяемый входной двумерный массив `A` – исходная квадратная матрица `A`, а также ссылки на выходные двумерные массивы `Q` и `R`.

Имплементации этого интерфейса используют для разложения либо отражения Хаусхолдера (лежат в директории `HouseholderMethod`), либо вращения Гивенса (соответственно, в `GivensMethod` директории). Классы (например, `HouseholderBasic.h`, и др.) разработаны в процессе итеративной оптимизации базовой реализации QR-разложения с использованием одного из двух этих методов. Порядок оптимизации следующий:

- «Семейство» классов на основе метода Хаусхолдера:
 - `HouseholderBasic.h` содержит базовую версию разложения с использованием отражений Хаусхолдера.
 - `HouseholderWithoutMatrixMultiplication.h` — вторая реализация, избавленная от использования матричных умножений.
 - `HouseholderWithNormWInplace.h` — третья реализация, использующая `in-place` хранение векторов вращения.
- «Семейство» классов на основе метода Гивенса:
 - `GivensBasic.h` содержит базовую версию разложения с использованием вращений Гивенса.
 - `GivensQRInOneStruct.h` — вторая реализация, в которой произведено улучшение кэш-локальности матриц `Q` и `R`.
 - `GivensVectorized.h` — третья реализация, использующая векторизацию.

Конечный пользователь может единообразно использовать их, согласно принципу полиморфизма – одной из парадигм Объектно-Ориентированного языка, коим является C++, т.е. создав экземпляр очередного класса оптимизации как указатель на базовый класс `IQRSolver`.

Структура данных для матрицы — это шаблонный вектор векторов из стандартной библиотеки: `std::vector<std::vector<T>>`. Это позволяет удобно работать с изменяемым параметром алгоритма «тип данных» и передать управление программной памятью методам класса, так как:

- `std::vector` может быть инициализирован с использованием шаблонного параметра `float` или `double`,
- `std::vector` самостоятельно контролирует аллокацию и освобождение памяти.

Разработана также небольшая шаблонная статическая библиотека `Matrix/MatrixOperations.h`, в которой реализованы некоторые алгебраические операции (сложение, вычитание, умножение, транспонирование матриц), а также функции для генерации матриц заданных форматов (произвольная или хессенбергова) со случайными числами, вывода данных на экран и в файл и др.

Для работы с транспонированной матрицей внутри статической шаблонной библиотеки `Matrix/TransposedMatrix.h` представлен одноимённый шаблонный класс. Матрица также представлена стандартным вектором из векторов: `std::vector<std::vector<T>>`.

Некоторые оптимизированные версии алгоритмов используют для представления матрицы небольшие модификации представленного выше типа данных:

- стандартный вектор длины $N \times N$, матрица записана по строкам (матрица в ленточном виде);
- `TransposedMatrix` — транспонированная матрица, записанная по столбцам. В основе такой же стандартный вектор длины $N \times N$;
- `QRMatrix` – матрица, хранящая в себе элементы структуры для одновременного доступа к одному элементу Q и R в одной и той же позиции для оптимизации обращения к аппаратному кэшу.

Файл `main.cpp` содержит примеры использования структур данных и классов-оптимизаций QR-разложения из разработанной библиотеки.

- Различные тестовые сценарии для разложения произвольных матриц A размером $N \times N$ с указанным типом данных,
- а также сценарии для разложения хессенберговых верхних и нижних матриц A размером $N \times N$ с указанным типом данных.

Для всех тестов выводятся на экран:

1. Размер задачи: test size: N
2. Время выполнения: time: <число в секундах>
3. Абсолютная ошибка: abs error: <вещественное число, $\|QR - A\|_F$ >

4. Относительная ошибка: rel error: <вещественное число, $\frac{\|QR - A\|_F}{\|A\|_F}$ >

При этом точность чисел с плавающей запятой, float или double, указывается через тип `currentType`. Тут стоит отметить, что в ходе работы была выявлена проблема быстрого накопления ошибки для типа данных, представляющих в C++ числа с плавающей точкой одинарной точности, float. Ошибка вела себя нестабильно, различные методы корректировки точности не помогали. Изменение на double, тип данных для числа с плавающей точкой двойной точности, дало стабильность для порядка абсолютной ошибки — не менее 10 знаков после запятой вне зависимости от размеров данных. При этом различные версии работают без серьёзного замедления даже на данных более 1000x1000 элементов. Шаблонный код позволяет легко управлять этим параметром через механизм создания псевдонимов для типов данных:

```
typedef double currentType;
```

Другой параметр для тестирования алгоритмов, количество потоков (по умолчанию равный максимальному количеству логических ядер в системе, где велась разработка):

```
size_t thread_num = 8;
```

Для проверки корректности работы программы в директории `test_data` хранится скрипт для Scilab, который:

- 1 Считывает из файла ту же матрицу A, над которой производила расчёты тестируемая библиотека.
- 2 Вычисляет для неё QR-разложение и подсчитывает время разложения.
- 3 Считывает вычисленные тестируемой программой и записанные в текстовые файлы матрицы Q и R.
- 4 Подсчитывает для каждой абсолютную и относительные ошибки по сравнению с полученным в скрипте разложением.
- 5 Выводит данные на экран в формате
 - 5.1 Размер задачи: test size: N
 - 5.2 Время выполнения: time: <число в секундах>
 - 5.3 Абсолютная ошибка для матрицы A: abs error: <вещественное число, $\|scilab_Q * scilab_R - A\|_F$ >
 - 5.4 Относительная ошибка для матрицы A: rel error: <вещественное число, $\frac{\|scilab_Q * scilab_R - A\|_F}{\|A\|_F}$ >
 - 5.5 Абсолютная ошибка для предвычисленной матрицы Q: abs error: <вещественное число, $\|my_Q - scilab_Q\|_F$ >

5.6 Относительная ошибка для предвычисленной матрицы Q: rel error: <вещественное

число, $\frac{\|my_Q - scilab_Q\|_F}{\|scilab_Q\|_F} >$

5.7 Абсолютная ошибка для предвычисленной матрицы R: abs error: <вещественное

число, $\|my_R - scilab_R\|_F >$

5.8 Относительная ошибка для предвычисленной матрицы R: rel error: <вещественное

число, $\frac{\|my_R - scilab_R\|_F}{\|scilab_R\|_F} >$

2.2 Реализация метода отражений Хаусхолдера

2.1.2 Первая реализация

С помощью ортогональных преобразований приведём матрицу A к треугольному виду. Возьмём первый столбец матрицы A : $(a_{11}, a_{21}, \dots, a_{n1})^T$. Построим матрицу Хаусхолдера $H_k = E - 2v_k v_k^T$ с помощью вектора $v_1 = \mu_1(a_{11} - \beta_1, a_{21}, \dots, a_{n1})^T$ и скаляров $\beta_1 = \text{sign}(-a_{11})\sqrt{\sum_{i=1}^n a_{i1}^2}$ и $\mu_1 = \frac{1}{\sqrt{2\beta_1^2 - 2\beta_1 a_{11}}}$. И, применив её к матрице, получим матрицу $\tilde{A}_1 = H_1 A$ со столбцом нулей под первым диагональным элементом, где $\tilde{a}_{11} = \beta_1$.

На втором этапе нужно поступить таким же образом с подматрицей матрицы, которая получается вычеркиванием в первой строки и первого столбца. Легко проверить, что это равносильно применению ко всей матрице преобразования Хаусхолдера, определяемого формулами $H_2 = E - 2v_2 v_2^T$, $v_2 = \mu_2(0; \tilde{a}_{22} - \beta_2, \tilde{a}_{32}, \dots, \tilde{a}_{n2})^T$, $\beta_2 = \text{sign}(-\tilde{a}_{22})\sqrt{\sum_{i=2}^n \tilde{a}_{i2}^2}$, $\mu_2 = \frac{1}{\sqrt{2\beta_2^2 - 2\beta_2 \tilde{a}_{22}}}$.

Таким образом, чтобы занулить $N-1$ строк матрицы A размером $N \times N$, требуется выполнить $N-1$ шагов. На каждом из них нужно вычислять матрицу отражения H :

1. $\beta_k = \text{sign}(-\tilde{a}_{kk})\sqrt{\sum_{i=k}^n \tilde{a}_{ik}^2}$
2. $\mu_k = \frac{1}{\sqrt{2\beta_k^2 - 2\beta_k \tilde{a}_{kk}}}$
3. $v_k = \mu_k(0; \dots; 0; \tilde{a}_{kk} - \beta_k; \tilde{a}_{(k+1)k}; \dots; \tilde{a}_{nk})$, первые $(k-1)$ позиций – нулевые.
4. $H_k = E - 2v_k v_k^T$

Матрица $\tilde{A}_k = H_{k-1} \dots H_1 A$, т.е. на каждом шаге алгоритма домножаем матрицу A слева на новую матрицу отражения и производим следующие вычисления уже над ней. При этом матрицу $\tilde{A}_k$ можно не хранить отдельно, по сути таким образом накапливается матрица R : $R = H_{n-1} \dots H_1 A$. Для простейшей реализации матрицу Q храним и вычисляем отдельно на каждом шаге как произведение всех матриц отражения: $Q_k = H_1 \dots H_k$. В результате получим $Q = H_1 \dots H_{n-1}$.

Первая реализация метода Хаусхолдера имеет существенные недостатки:

- два матричных произведения на каждую из $(N-1)$ итераций цикла,
- отдельное хранение матрицы отражения H ($N \times N$ элементов) для вычисления Q .

На каждом шаге алгоритма получается вектор Хаусхолдера v и матрица отражения H :

$$H = E - \frac{vv^T}{v^T v}$$

Домножая матрицу A слева на матрицы отражения H_k , получим матрицу R :

$$A_k = H_{k-1} \dots H_1 A,$$

$$R = H_{n-1} \dots H_1 A$$

Матрица Q при этом хранится и также вычисляется на каждом шаге с помощью последовательных домножений справа всех матриц отражения:

$$Q = H_1 \dots H_{n-1}$$

Программная реализация с учётом описанного выше алгоритма будет следующей. Копируем матрицу A в R . В цикле от 0 до $N-1$ подсчитываем по формулам β, μ, v и H . Подсчитываем Q как перемножение матриц отражения друг на друга, домножаем R на H слева и получаем матрицу для следующей итерации:

```

Процедура QR_decomposition(A, Q, R):
R = A // копирование исходной матрицы в R
Q = I // Q инициализируется как единичная
Для k от 0 до N-1 выполнить: // цикл по столбцам
    // подсчёт  $\beta$ 
    sum = 0
    Для i от k до N-1:
        sum = sum + R[i][k] * R[i][k]
    Конец цикла
     $\beta = \text{sign}(-R[k][k]) * \text{sqrt}(\text{sum})$ 

    // подсчёт  $\mu$ 
     $\mu = 1.0 / \text{sqrt}(2.0 * \beta * (\beta - R[k][k]))$ 

    // построение  $v[N]$ 
    v = (0, ..., 0), len(v) = N
    Для i от 0 до N-1:
        v[i] = R[i][k] *  $\mu$ 
    Конец цикла
    v[k] = v[k] -  $\beta * \mu$ 

    // построение матрицы Хаусхолдера H
    H = I - 2 * v * v^T

    R = H * R // построение матрицы R
    Q = Q * H // построение матрицы Q
Конец цикла
Конец процедуры

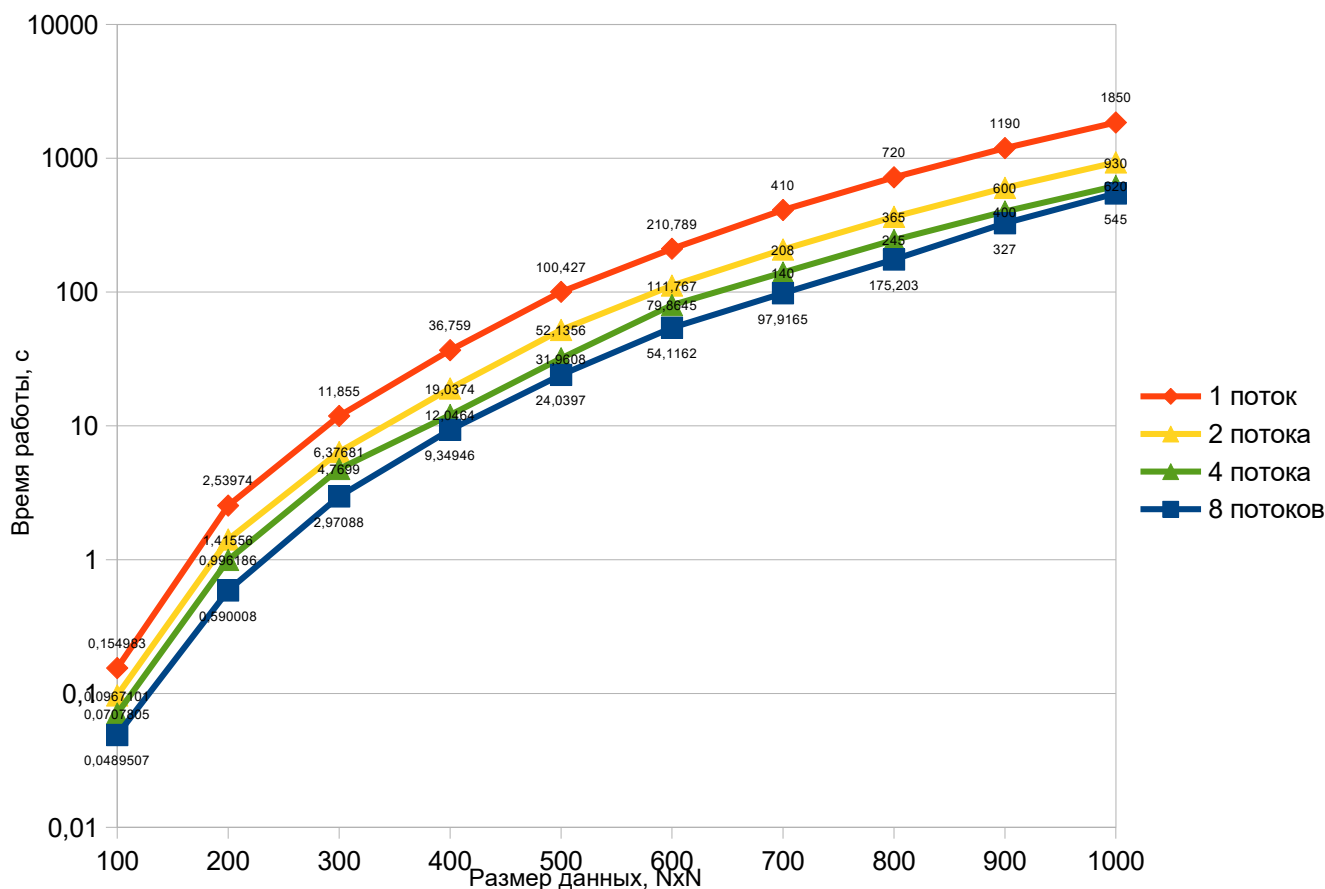
```


Эта реализация полностью соответствует описанному ранее теоретическому алгоритму, но долго работает. Получить параллельную версию можно с помощью добавления параллельного блочного алгоритма умножения матриц и использования параллелизма при вычислении вектора Хаусхолдера v :

- Для циклов подсчёта вектора Хаусхолдера v ($v_k = \mu_k(0; \dots; 0; \tilde{a}_{kk} - \beta_k; \tilde{a}_{(k+1)k}; \dots; \tilde{a}_{nk})$) и матрицы отражения H ($H = E - \frac{vv^T}{v^T v}$) воспользоваться прагмой OpenMP для расспараллеливания циклов `#pragma omp parallel for` с параметром `num_threads(thread_num)`, который можно менять для удобства замеров времени.
- Для подсчёта суммы квадратов по столбцу в рамках расчёта β ($\beta_k = \text{sign}(-\tilde{a}_{kk}) \sqrt{\sum_{i=k}^n \tilde{a}_{ik}^2}$) воспользоваться клаузой для прагмы параллельного вычисления циклов `reduction(+:sum)`.

Замеры для типа данных `double` (вещественное число двойной точности) на разном количестве потоков дали следующие результаты:

Схема 1 – Замеры времени работы программы на разных размерах матрицы A



Полученное ускорение в виде отношения времени работы программы, запущенной в параллельном режиме, ко времени работы последовательной версии для различного количества потоков можно посмотреть в таблице ниже:

Таблица 1 – Данные об ускорении для первой реализации QR-разложения с помощью отражений Хаусхолдера в зависимости от количества потоков

Размер данных, NxN	Время работы последовательной версии	Ускорение относительно последовательной версии		
		2 потока	4 потока	8 потоков
100	0,154983	1,6	2,19	3,17
200	2,53974	1,79	2,55	4,3
300	11,855	1,86	2,49	3,99
400	36,759	1,93	3,05	3,93
500	100,427	1,93	3,14	4,18
600	210,789	1,89	2,64	3,9
700	≈ 410	1,97	2,93	4,19
800	≈ 720	1,97	2,94	4,11
900	≈ 1190	1,98	2,98	3,64
1000	≈ 1850	1,99	2,98	3,39
СРЕДНЕЕ ПОЛУЧЕННОЕ УСКОРЕНИЕ	-	1,89	2,79	3,88

Видно, что для двух потоков получилось ускорить решение в два раза, в то время как для четырёх и восьми потоков дела обстоят хуже. Хорошее масштабирование для двух потоков, накладные расходы на создание и поддержание потоков минимальны. Пик ускорения достигается при N=500-800, затем наблюдается спад из-за кэш-промахов. Линейного ускорения тут добиться не получилось из-за того, что повышение накладных расходов перекрывает эффект от параллельного исполнения операций. Вероятно, все упирается в обращения к памяти — разные потоки обращаются к разным участкам, не влезаящим в один кэш.

Абсолютная ошибка для типа числа с плавающей точкой двойной точности вне зависимости от размера данных имеет порядок $1e-12$, относительная — $1e-15$.

Реализация прикреплена в Приложении 1.

2.1.2 Избавление от матричных умножений

Напрямую перемножать матрицы $N \times N$ на каждой итерации очень неэффективно, подсчитаем перемножение $H_k A_k$ по-другому:

$$H_k A_k = (I - 2vv^T) A_k = A_k - 2v(v^T A_k),$$

получим более дешёвые операции над векторами и матрицей A , уменьшим сложность в N раз.

Также вычисление R этим способом позволяет упростить вычисление Q . Для неё понадобится только сохранить отдельно вектора Хаусхолдера v_k . Вычислять Q будем восстанавливая матрицы отражения из сохранённых векторов w , в обратном порядке:

$$\begin{aligned} Q_{n-1} &= I, \\ Q_k &= H_k Q_{k-1} = Q_{k-1} - 2v_k(v_k^T Q_{k-1}), k = n-2, \dots, 0, \\ Q &= Q_0 \end{aligned}$$

Получим экономию памяти порядка N и ускорение времени больше, чем на порядок.

В данной реализации используется ленточное представление матриц (в виде одномерных массивов, в которые матрицы записаны по строкам) или в виде плоского массива (одномерный массив, в который рекурсивно объединены элементы вложенных подмассивов до указанного уровня вложенности). Все вектора Хаусхолдера складываются в `all_v` одномерный массив длины $N \times (N-1)$ (незначащие k нулей не хранятся) для вычисления Q . Вычисление матрицы R выполняется с сохранением векторов в этот массив. Матрица Q строится обратным применением сохранённых преобразований к единичной матрице. Для ускорения вычислений используются предварительно вычисленные смещения kN и указатели на столбцы матрицы. Алгоритм использует параллелизм OpenMP только для больших матриц ($N \geq 1000$). Итоговые результаты конвертируются обратно в вектор вложенных векторов.

Программная реализация с учётом описанного выше алгоритма будет следующей. Перевод данных в плоский вид. Инициализируем массив для хранения векторов Хаусхолдера. В цикле от 0 до $N-1$ подсчитываем по формулам β, μ, v и H . Домножаем R на H слева и получаем матрицу для следующей итерации. Сохраняем v в массив векторов. Подсчитываем Q с помощью сохранённых отражений в обратном порядке:

```

Процедура QR_decomposition(A, Q, R):
R_flat = A_flat // копирование исходной матрицы в R
Q_flat = I // Q инициализируется как единичная
all_v = матрица Nx(N-1), инициализированная 0
Для k от 0 до N-1 выполнить: // цикл по столбцам
    v = ссылка на all_v[k]
    // подсчёт  $\beta$  из предыдущего кода
    // подсчёт  $\mu$  из предыдущего кода
    // построение  $v[N]$  из предыдущего кода

    // отражение
    //  $R = R - 2 * v * (v^T * R)$ 
    // предварительно вычисляем  $2*v^T R$  для каждого столбца
    _2vTR = вектор(длина(v))
    Для j от k до N-1 выполнить:
        vTR = 0
        Для i от 0 до длина(v) выполнить:
            vTR = vTR + v[i] * R_flat[i*N + j]
        Конец цикла
        _2vTR[j-k] = 2 * vTR
    Конец цикла

    // обновление R
    //  $R = R - v * 2v^T R$ 
    Для j от k до N-1 выполнить:
        Для i от 0 до длина(v) выполнить:
            R_flat[i*N + j] = R_flat[i*N + j] - v[i] * _2vTR[j - k]
        Конец цикла
    Конец цикла
Конец цикла

```

```

// вычисление Q
// начать с  $Q_{N-1} = H_{N-2} * I = I - 2 * v_{N-2} * (v_{N-2}^T * I)$  и домножать слева на
матрицы отражения
//  $Q = H_1 * \dots * (H_k * Q_{k-1}) = H_1 * \dots * (Q_{k-1} - 2 * v_k * (v_k^T * Q_{k-1}))$ 
Для k от N-2 до 0 выполнить:
    v = ссылка на all_v[k]
    //  $2v^TQ$  параллельно по столбцам
    _2vTQ = вектор(N) инициализированный 0
    Для j от 0 до N-1 выполнить:
        sum = 0
        Для i от 0 до длина(V) выполнить:
            sum = sum + v[i] * Q_flat[(k + 1) * N + j]
        Конец цикла
    _2vTQ = 2 * sum
    Конец цикла

    //  $Q - v * 2v^TQ$  параллельно по строкам
    Для i от 0 до длина(V) выполнить:
        Для j от 0 до N-1 выполнить:
            Q_flat[(k + 1) * N + j] = Q_flat[(k + 1) * N + j] - v[i] * _2vTQ[j]
        Конец цикла
    Конец цикла
Конец цикла
R = R_flat
Q = Q_flat
Конец процедуры

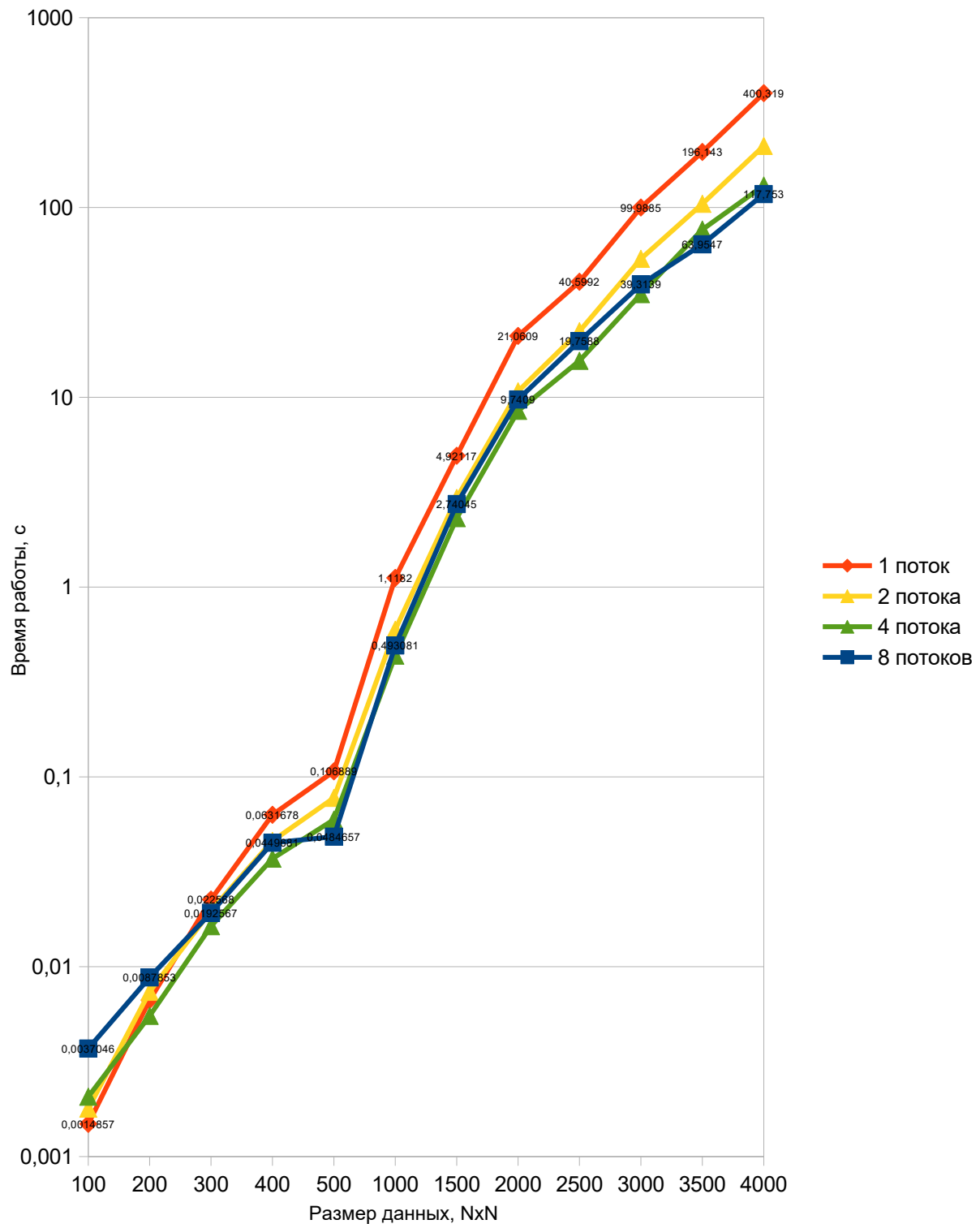
```

Использование параллелизма:

- при вычислении β, μ и вектора Хаусхолдера в цикле с помощью `#pragma omp parallel for` и с редукцией при суммировании элементов: `reduction(+:sum)`
- отражение в цикле по строкам при вычислении R и Q можно также выполнять параллельно.

Замеры для типа данных `double` (вещественное число двойной точности) на разном количестве потоков дали следующие результаты:

Схема 2 – Замеры времени работы программы на разных размерах матрицы



Полученное ускорение в виде отношения времени работы программы, запущенной в параллельном режиме, ко времени работы последовательной версии для различного количества потоков можно посмотреть в таблице ниже:

Таблица 2 – Данные об ускорении для второй реализации QR-разложения с помощью отражений Хаусхолдера в зависимости от количества потоков

Размер данных, NxN	Время работы последовательной версии	Ускорение относительно последовательной версии		
		2 потока	4 потока	8 потоков
100	0,0014857	0,83	0,72	0,4
200	0,0066652	0,9	1,21	0,76
300	0,022568	1,14	1,38	1,17
400	0,0631678	1,37	1,7	1,4
500	0,106889	1,38	1,79	2,21
1000	1,1182	1,87	2,57	2,27
1500	4,92117	1,67	2,13	1,8
2000	21,0609	1,96	2,48	2,16
2500	40,5992	1,82	2,61	2,05
3000	99,9885	1,86	2,86	2,54
3500	196,143	1,87	2,57	3,07
4000	400,319	1,9	3,06	3,4
СРЕДНЕЕ ПОЛУЧЕННОЕ УСКОРЕНИЕ	-	1,55	2,09	1,94

В этой версии не удаётся добиться линейного ускорения, видно, что при $N \leq 300$ использование нескольких потоков замедляет вычисления (ускорение становится < 1), так как накладные расходы на создание потоков и их синхронизацию превышают время самих вычислений. Для небольших данных можно было бы отключить распараллеливание. Возможно, улучшения можно добиться путём объединения нескольких циклов в один для избежания излишней синхронизации. Может быть улучшен не совсем оптимальный доступ к памяти при обновлении R с шагом в N элементов, что ухудшает кэш-локальность алгоритма. Избыточное выделение памяти под промежуточные вектора может также создавать дополнительные задержки для больших N.

Абсолютная ошибка для типа числа с плавающей точкой двойной точности вне зависимости от размера данных имеет порядок $1e-12$, относительная — $1e-15$.

Реализация прикреплена в Приложении 2.

2.1.3 Работа с нормированными векторами Хаусхолдера

Если изменить способ вычисления вектора Хаусхолдера v_k :

$$\sigma = -\text{sign}(a_k[0]) \|a_k\|,$$

$$v_k = a_k - \sigma e_k = \begin{bmatrix} 0; \dots; 0; \underbrace{a_{kk} - \sigma}_{k-1}; a_{k,k+1}; \dots; a_{k,n} \end{bmatrix},$$

$$H = I - \tau * (v v^T), \text{ где } \tau = \frac{2}{v^T v},$$

$$H_k A_k = (I - \tau v v^T) A_k = A_k - \tau v (v^T A_k)$$

его можно сохранить под диагональю в матрице A_k вместо незначащих нулей следующим образом:

$$H_1 A = A_1 = \begin{bmatrix} \sigma_1 & \widetilde{A}_{12} & \dots & \widetilde{A}_{1n} \\ v_{11}/v_{10} & \widetilde{A}_{22} & \dots & A_{2n} \\ \dots & \dots & \dots & \dots \\ v_{1k}/v_{10} & \widetilde{A}_{k2} & \dots & \widetilde{A}_{kn} \\ v_{1,k+1}/v_{10} & \widetilde{A}_{k+1,2} & \dots & \widetilde{A}_{k+1,n} \\ \dots & \dots & \dots & \dots \\ v_{1n}/v_{10} & \widetilde{A}_{n2} & \dots & \widetilde{A}_{n,n} \end{bmatrix},$$

$$H_2 A_1 = A_2 = \begin{bmatrix} \sigma_1 & \widetilde{\widetilde{A}}_{12} & \dots & \widetilde{\widetilde{A}}_{1n} \\ v_{11}/v_{10} & \sigma_2 & \dots & A_{2n} \\ v_{12}/v_{10} & v_{21}/v_{20} & \dots & A_{2n} \\ \dots & \dots & \dots & \dots \\ v_{1k}/v_{10} & v_{2k}/v_{20} & \dots & \widetilde{\widetilde{A}}_{kn} \\ v_{1,k+1}/v_{10} & v_{2,k+1}/v_{20} & \dots & \widetilde{\widetilde{A}}_{k+1,n} \\ \dots & \dots & \dots & \dots \\ v_{1n}/v_{10} & v_{2,n}/v_{20} & \dots & \widetilde{\widetilde{A}}_{n,n} \end{bmatrix},$$

$$H_k A_{k-1} = \begin{bmatrix} \sigma_1 & \widetilde{A}_{12} & \dots & \widetilde{A}_{1k} & \dots & \dots & \widetilde{A}_{1n} \\ v_{11}/v_{10} & \sigma_2 & \dots & \widetilde{A}_{2k} & \dots & \dots & A_{2n} \\ \dots & \dots & \ddots & \dots & \dots & \dots & \dots \\ v_{1k}/v_{10} & \dots & \dots & \sigma_k & \dots & \dots & \widetilde{A}_{kn} \\ v_{1,k+1}/v_{10} & \dots & \dots & v_{k1}/v_{k0} & \widetilde{A}_{k+1,k+1} & \dots & \widetilde{A}_{k+1,n} \\ \dots & \dots & \dots & \vdots & \vdots & \ddots & \dots \\ v_{1n}/v_{10} & \dots & \dots & v_{kn}/v_{k0} & \widetilde{A}_{n,k+1} & \dots & \widetilde{A}_{n,n} \end{bmatrix}$$

Такое использование данных называется in-place алгоритмом. Такие алгоритмы модифицируют входные данные непосредственно в месте их исходного хранения, без выделения дополнительной памяти. Это позволяет повысить эффективность работы с

памятью при вычислении над большими объёмами данных или в условиях ограниченных ресурсов.

Отражение заключается в применении к столбцам справа от обнуляемого, от $(k+1)$ до $(N-1)$, преобразования $R - \tau v(v^T R)$, где $(v^T R)$ — скалярное произведение вектора Хаусхолдера из столбца col , вычитание из R выполняется над тем же столбцом.

Отдельно вычисляется матрица Q с помощью сохранённых под диагоналями нормированных векторов Хаусхолдера, как в предыдущей реализации, только вместо 2 используется $\tau = \frac{2}{v^T v}$ в качестве коэффициента. Работа также идёт только над столбцами, от k до N : скалярное умножение с вектором и вычитание вектора из столбца.

Третья версия QR-разложения методом Хаусхолдера представляет собой дальнейшую оптимизацию алгоритма, направленную на повышение эффективности работы с памятью. Данная версия использует класс для транспонированной матрицы `TransposedMatrix`, что позволяет быстрее работать со столбцам. По сравнению с предыдущей версией векторы Хаусхолдера не хранятся отдельно, а сохраняются в поддиагональных элементах матрицы R .

Вычисление отражений Хаусхолдера производится новым способом: вектор Хаусхолдера v_k формируется таким образом, чтобы первый элемент становился равным единице. Эта нормировка помогает уменьшить погрешность, не хранить $(k+1)$ -ый элемент явно.

Применение отражений к правой части матрицы R и построение матрицы Q оптимизированы за счёт более быстрого обращения к столбцам.

Транспонирование осуществляется средствами класса `TransposedMatrix`.

Программная реализация с учётом описанного выше алгоритма будет следующей. Перевод данных в плоский и транспонированный вид. Инициализируем массив для хранения векторов Хаусхолдера. В цикле от 0 до $N-1$ подсчитываем по формулам σ, v, τ . Домножаем R на H слева и получаем матрицу для следующей итерации. Сохраняем v в R под диагональю. Подсчитываем Q с помощью сохранённых отражений в обратном порядке:

```

Процедура QR_decomposition(A, Q, R):
R_T = A // копирование исходной матрицы в R
Q_T = I // Q инициализируется как единичная
Для k от 0 до N-1 выполнить: // цикл по столбцам
    // подсчёт  $\sigma$ 
    sum = 0
    Для i от k до N-1 выполнить:
        sum = sum + R_T[i][k] * R_T[i][k]
    Конец цикла
     $\sigma = -\text{sign}(R_T[k][k]) * \text{sqrt}(\text{sum})$ 

    // подсчёт v
    // копирование в буфер v
    v = вектор(N-k)
    Для i от 1 до N-k-1 выполнить:
        v[i] = R_T[k+i][k]
    Конец цикла
    v[0] = R_T[k][k] -  $\sigma$ 

    // подсчёт  $\tau$ 
    sum = 0
    Для i от 0 до N-k-1 выполнить:
        sum = sum + v[i] * v[i]
    Конец цикла
     $\tau = 2 / \text{sum}$ 

    // сохраняем нормированный v для Q (v[0]=1) в R под диагональю
    scale = 1 / v[0]
    Для i от 1 до N-k-1 выполнить:
        R_T[k+i][k] = v[i] * scale
    Конец цикла
    R_T[k][k] =  $\sigma$ 

    // отражение
    Для col от k+1 до N-1 выполнить:
        vTR = 0
        Для i от 0 до длина(v) выполнить:
            vTR = vTR + v[i] * R_T[k + i][col]
        Конец цикла
        scale =  $\tau * \text{vTR}$ 
        // обновляем столбец col: R[:,col] -= v * scale
        Для i от 0 до длина(v) выполнить:
            R_T[k + i][col] = R_T[k + i][col] - v[i] * scale
        Конец цикла
    Конец цикла
Конец цикла

```

```

// вычисление Q
// применяем отражения в обратном порядке
Для k от N-2 до 0 выполнить:
    // берём вектор v из R
    v = вектор(N-k), v[0] = 1
    Для i от 1 до длина(v) выполнить:
        v[i] = R_T[k + i][k]
        R_T[k + i][k] = 0
    Конец цикла

    // подсчёт  $\tau$ 
    sum = 0
    Для i от 0 до N-k-1 выполнить:
        sum = sum + v[i] * v[i]
    Конец цикла
     $\tau = 2 / \text{sum}$ 

    //  $H = I - \tau * v * v^T$  к Q[k:N, :]
    Для j от 0 до N-1 выполнить:
        //  $v^T Q = v^T * Q[k:N, \text{col}]$ 
        vTQ = Q_T[k][j] // v[0] = 1
        Для i от 1 до длина(v) выполнить:
            vTQ = vTQ + v[i] * Q_T[k+i][j]
        Конец цикла
        vTQ = vTQ *  $\tau$ 

        //  $Q[k][j] -= v^T Q * v$ 
        Q_T[k][j] = Q_T[k][j] - vTQ
        Для i от 1 до длина(v) выполнить:
            Q_T[k+i][j] = Q_T[k+i][j] - vTQ * v[i]
        Конец цикла
    Конец цикла
Конец цикла
R = транспонировать(R_T)
Q = транспонировать(Q_T)
Конец процедуры

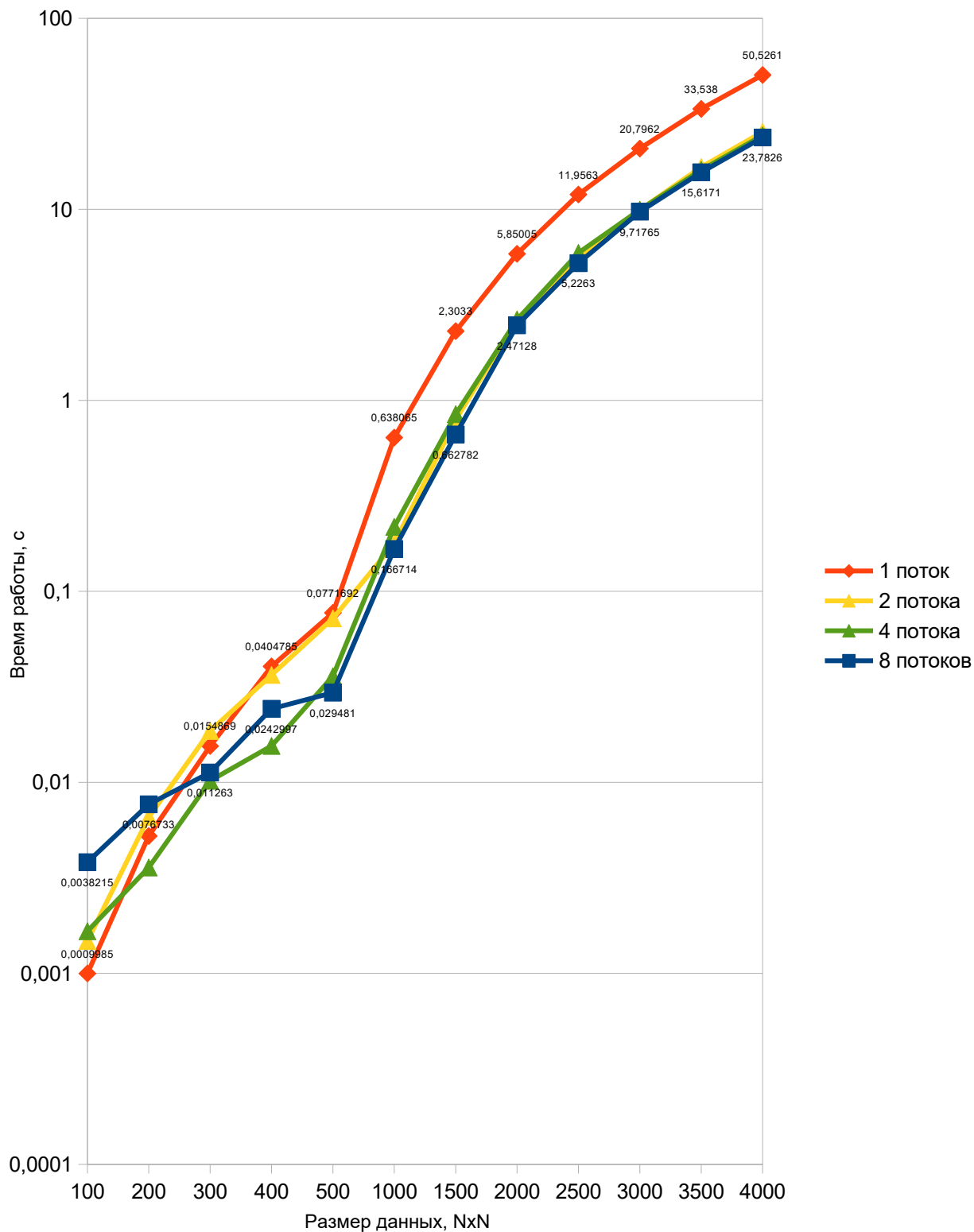
```

Использование параллелизма:

- при вычислении σ , τ (`reduction(+:sum)`) и вектора Хаусхолдера v в цикле с помощью `#pragma omp parallel for`
- отражение в цикле по строкам при вычислении R и Q можно также выполнять параллельно.

Замеры для типа данных double (вещественное число двойной точности) на разном количестве потоков дали следующие результаты:

Схема 3 – Замеры времени работы программы на разных размерах матрицы A



Полученное ускорение в виде отношения времени работы программы, запущенной в параллельном режиме, ко времени работы последовательной версии для различного количества потоков можно посмотреть в таблице ниже:

Таблица 3 – Данные об ускорении для третьей реализации QR-разложения с помощью отражений Хаусхолдера в зависимости от количества потоков

Размер данных, N×N	Время работы последовательной версии	Ускорение относительно последовательной версии		
		2 потока	4 потока	8 потоков
100	0,0009985	0,68	0,6	0,26
200	0,0052362	0,8	1,46	0,68
300	0,0154869	0,83	1,52	1,38
400	0,0404785	1,11	2,61	1,67
500	0,0771692	1,07	2,14	2,62
1000	0,638065	3,58	2,93	3,83
1500	2,3033	2,91	2,72	3,48
2000	5,85005	2,2	2,21	2,37
2500	11,9563	2,16	2,02	2,29
3000	20,7962	2,09	2,09	2,14
3500	33,538	2,02	2,09	2,15
4000	50,5261	1,99	2,07	2,12
СРЕДНЕЕ ПОЛУЧЕННОЕ УСКОРЕНИЕ	-	1,79	2,04	2,08

Полученная версия быстрее при последовательном исполнении, но хуже параллелизуется. Получено отрицательное ускорение для малых N. По представленным выше данным заметно, как для $N \leq 200$ использование 8 потоков замедляет работу в 2-4 раза. Это говорит о необходимости отказаться от распараллеливания для таких размеров. Возможные улучшения: отказ от многократного создания векторов на каждой итерации, выделение единого буфера до начала цикла; использование отдельного класса TransposedMatrix и метода для доступа к данным может быть причиной существенного замедления; также можно объединить отдельные циклы для их совместной параллелизации.

Абсолютная ошибка для типа числа с плавающей точкой двойной точности вне зависимости от размера данных имеет порядок $1e-12$, относительная — $1e-15$.

Реализация прикреплена в Приложении 3.

Сводные данные ниже содержат сравнение всех полученных реализаций QR-разложения на основе отражений Хаусхолдера с первой последовательной реализацией. Последним столбцом прикреплено время работы QR-разложения из библиотеки SciLab. Условия исполнения: максимально доступное количество потоков в тестируемой системе (восемь), произвольная плотная матрица A , тип данных матрицы A – double, число s плавающей точкой двойной точности.

Схема 4 – Сравнение реализаций QR-разложения методом Хаусхолдера с QR-разложением из библиотеки SciLab

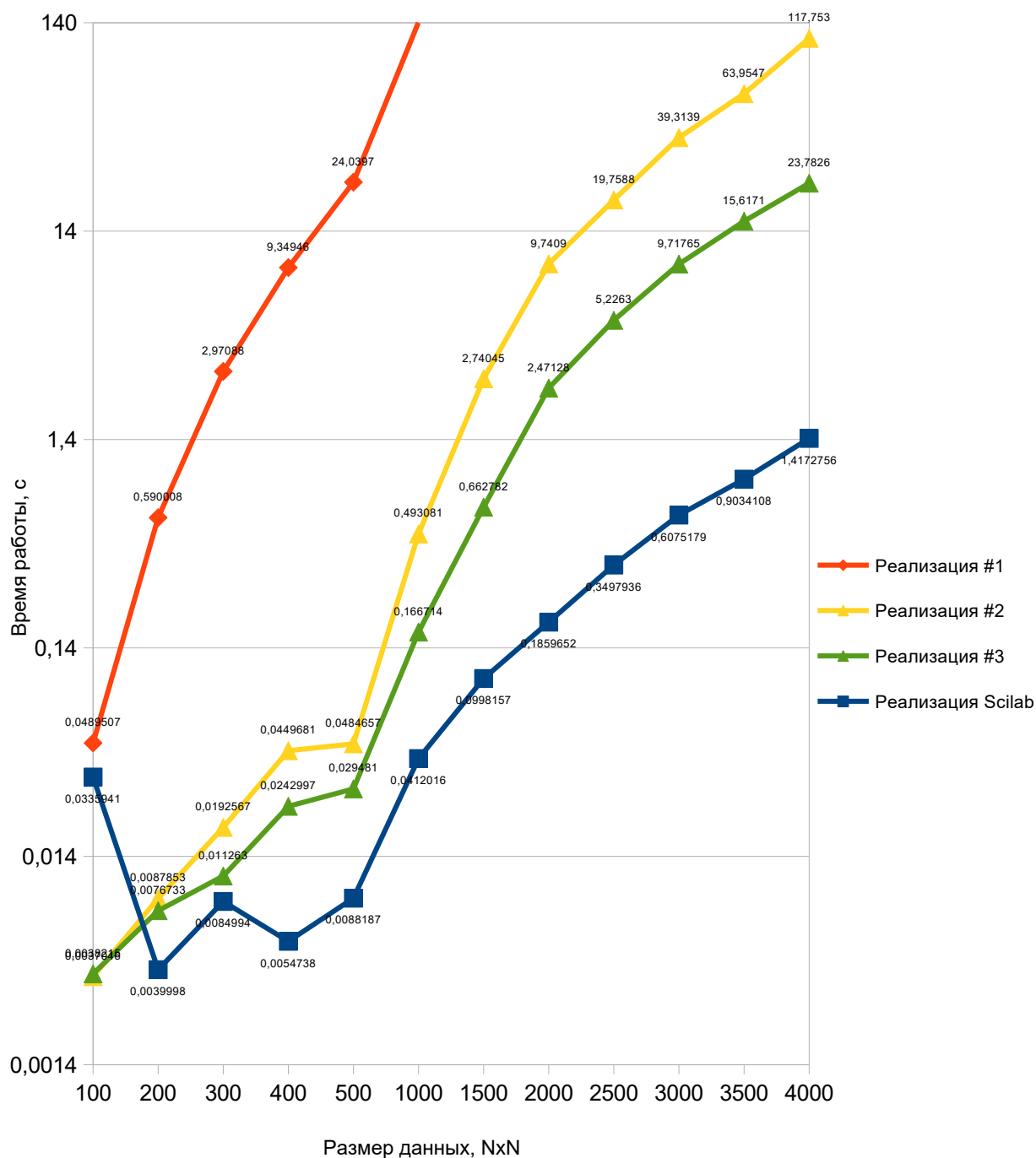


Таблица 4 – Сравнение версий метода Хаусхолдера и версии из библиотеки SciLab

Размер данных (матрица NxN)	Время работы первой реализации QR- разложения на основе отражений Хаусхолдера, с	Время работы второй реализации, без матричных умножений с	Время работы третьей реализации, in- place алгоритм с	Время работы QR- разложения из библиотеки SciLab, с
100	0,0489507	0,0037046	0,0038215	0.0335941
200	0,590008	0,0087853	0,0076733	0.0039998
300	2,97088	0,0192567	0,011263	0.0084994
400	9,34946	0,0449681	0,0242997	0.0054738
500	24,0397	0,0484657	0,029481	0.0088187
1000	545	0,493081	0,166714	0.0412016
1500	-	2,74045	0,662782	0.0998157
2000	-	9,7409	2,47128	0.1859652
2500	-	19,7588	5,2263	0.3497936
3000	-	39,3139	9,71765	0.6075179
3500	-	63,9547	15,6171	0.9034108
4000	-	117,753	23,7826	1.4172756

2.2 Реализация метода вращений Гивенса

2.2.1 Первая реализация

Базовая реализация алгоритма разложения двумерной матрицы A в двумерные верхнетреугольную R и ортогональную Q будет следующим:

1. для всех диагональных элементов (i, j) по очереди слева направо сверху вниз вычисляется параметр вращения:

$$\cos\theta = c = \frac{a_{jj}}{\sqrt{a_{jj}^2 + a_{ij}^2}}, \sin\theta = s = \frac{-a_{ij}}{\sqrt{a_{jj}^2 + a_{ij}^2}}$$

2. матрица A домножается слева на матрицу вращения G_{ji} :

$$G_{ji} \dots G_{12} A =$$

$$\begin{bmatrix} 1 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \dots & \ddots & / & / & / & / & / & / & / & / & 0 \\ 0 & \dots & 1 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & c & 0 & \dots & 0 & -s & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ / & / & / & / & / & / & / & / & / & / & / \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & s & 0 & \dots & 0 & c & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ / & / & / & / & / & / & / & / & / & / & / \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 1 \end{bmatrix} \cdot \begin{bmatrix} a_{11} & \dots & a_{1,j-1} & a_{1j} & a_{1,j+1} & \dots & a_{1,i-1} & a_{1i} & a_{1,i+1} & \dots & a_{1n} \\ \dots & \ddots & / & / & / & / & / & / & / & / & / \\ 0 & \dots & a_{j-1,j-1} & a_{j-1,j} & a_{j-1,j+1} & \dots & a_{j-1,i-1} & a_{j-1,i} & a_{j-1,i+1} & \dots & a_{j-1,n} \\ 0 & \dots & 0 & a_{jj} & a_{j,j+1} & \dots & a_{j,i-1} & a_{ji} & a_{j,i+1} & \dots & a_{jn} \\ 0 & \dots & 0 & 0 & a_{j+1,j+1} & \dots & a_{j+1,i-1} & a_{j+1,i} & a_{j+1,i+1} & \dots & a_{j+1,n} \\ / & / & / & / & / & / & / & / & / & / & / \\ 0 & \dots & 0 & 0 & a_{i-1,j+1} & \dots & a_{i-1,i-1} & a_{i-1,i} & a_{i-1,i+1} & \dots & a_{i-1,n} \\ 0 & \dots & 0 & a_{ij} & a_{i,j+1} & \dots & a_{i,i-1} & a_{ii} & a_{i,i+1} & \dots & a_{in} \\ 0 & \dots & 0 & a_{i+1,j} & a_{i+1,j+1} & \dots & a_{i+1,i-1} & a_{i+1,i} & a_{i+1,i+1} & \dots & a_{i+1,n} \\ / & / & / & / & / & / & / & / & / & / & / \\ 0 & \dots & 0 & a_{n,j} & a_{n,j+1} & \dots & a_{n,i-1} & a_{ni} & a_{n,i+1} & \dots & a_{nn} \end{bmatrix}$$

Красным отмечена главная диагональ, ниже которой все элементы должны быть занулены. Зелёным отмечен текущий зануляемый элемент a_{ij} , все столбцы левее его уже занулены, также занулены все элементы того же столбца выше a_{ij} . Синим выделены $\cos\theta$ (G_{ji} и G_{ii}) и $\sin\theta$ (G_{ji} и G_{ij}) и строки, которые будут вращаться.

Для вычисления новых значений в строках A_j и A_i перемножаются строка G_j и столбцы A_k , $k \in [j, \dots, N]$ и G_i и столбцы A_k , $k \in [j, \dots, N]$, то есть при этом в строках A_j и A_i ненулевые значения будут только правее, начиная с j столбца, что уменьшает количество вычислений. Вычисление нового значения заключается в сложении двух умножений:

$$\widetilde{a}_{ik} = a_{jk} \cdot c - a_{ik} \cdot s,$$

$$\widetilde{a}_{jk} = a_{jk} \cdot s + a_{ik} \cdot c.$$

3. матрица Q вычисляется с помощью полученной матрицы вращения домножением на неё справа:

$$Q = G_{12}^T \dots G_{ji}^T$$

$$\begin{bmatrix} Q_{11} & \dots & Q_{1,j-1} & Q_{1j} & Q_{1,j+1} & \dots & Q_{1,i-1} & Q_{1i} & Q_{1,i+1} & \dots & Q_{1n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{j-1,1} & \dots & Q_{j-1,j-1} & Q_{j-1,j} & Q_{j-1,j+1} & \dots & Q_{j-1,i-1} & Q_{j-1,i} & Q_{j-1,i+1} & \dots & Q_{j-1,n} \\ Q_{j,1} & \dots & Q_{j,j-1} & Q_{jj} & Q_{j,j+1} & \dots & Q_{j,i-1} & Q_{ji} & Q_{j,i+1} & \dots & Q_{jn} \\ Q_{j+1,1} & \dots & Q_{j+1,j-1} & Q_{j+1,j} & Q_{j+1,j+1} & \dots & Q_{j+1,i-1} & Q_{j+1,i} & Q_{j+1,i+1} & \dots & Q_{j+1,n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{i-1,1} & \dots & Q_{i-1,j-1} & Q_{i-1,j} & Q_{i-1,j+1} & \dots & Q_{i-1,i-1} & Q_{i-1,i} & Q_{i-1,i+1} & \dots & Q_{i-1,n} \\ Q_{i,1} & \dots & Q_{i,j-1} & Q_{ij} & Q_{i,j+1} & \dots & Q_{i,i-1} & Q_{ii} & Q_{i,i+1} & \dots & Q_{in} \\ Q_{i+1,1} & \dots & Q_{i+1,j-1} & Q_{i+1,j} & Q_{i+1,j+1} & \dots & Q_{i+1,i-1} & Q_{i+1,i} & Q_{i+1,i+1} & \dots & Q_{i+1,n} \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ Q_{n,1} & \dots & Q_{n,j-1} & Q_{nj} & Q_{n,j+1} & \dots & Q_{n,i-1} & Q_{ni} & Q_{n,i+1} & \dots & Q_{nn} \end{bmatrix} \cdot \begin{bmatrix} 1 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 1 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & c & 0 & \dots & 0 & -s & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & s & 0 & \dots & 0 & c & 0 & \dots & 0 \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 1 & \dots & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 1 \end{bmatrix}$$

Красным отмечена главная диагональ. Синим выделены $\cos\theta$ (G_{jj} и G_{ii}) и $\sin\theta$ (G_{ji} и G_{ij}) и столбцы, которые будут вращаться.

Для вычисления новых значений в столбцах Q_j и Q_i перемножаются строка Q_k , $k \in [1, \dots, N]$ и столбцы G_j , и Q_k , $k \in [1, \dots, N]$ и столбцы G_i , количество вычислений здесь не уменьшается. Вычисление нового значения заключается в сложении двух умножений:

$$\widetilde{q}_{ki} = q_{kj} \cdot s + q_{ki} \cdot c,$$

$$\widetilde{q}_{kj} = q_{kj} \cdot c - q_{ki} \cdot s.$$

Базовая версия программы выполнена по следующему алгоритму:

1. Занулить $\left(\frac{N^2 - N}{2}\right)$ элементов, для этого в цикле по столбцам j , $j \in [0, N - 1]$ для всех строк i , $i \in [j + 10, N]$ выполнить вращения элемента (i, j) :

а) Проверить, нужно ли занулять элемент (i, j) . Если он уже близок к нулю, пропустить это вращение.

б) Вычислить c и s по формулам:

$$\cos\theta = c = \frac{a_{jj}}{\sqrt{a_{jj}^2 + a_{ij}^2}}, \sin\theta = s = \frac{-a_{ij}}{\sqrt{a_{jj}^2 + a_{ij}^2}}.$$

с) Вращать две строки i и j (элементы a_{jk} и a_{ik} , $k \in [i, N]$), выполнив вычисления по формулам:

$$\widetilde{a}_{ik} = a_{jk} \cdot c - a_{ik} \cdot s,$$

$$\widetilde{a}_{jk} = a_{jk} \cdot s + a_{ik} \cdot c.$$

Это можно сделать параллельно для всех k .

d) С помощью матрицы вращения накапливать матрицу Q. Для этого вращать два столбца i и j (элементы q_{kj} и q_{ki}, $k \in [0, N)$), выполнив вычисления по формулам:

$$\widetilde{q}_{ki} = q_{kj} \cdot s + q_{ki} \cdot c,$$

$$\widetilde{q}_{kj} = q_{kj} \cdot c - q_{ki} \cdot s.$$

К вычислению Q может быть два подхода.

Можно вычислять вращения столбцов для Q в том же месте, где вычисляется R. В результате код будет выглядеть следующим образом:

Процедура QR_decomposition(A, Q, R):

R = A // копирование исходной матрицы в R

Q = I // Q инициализируется единичной матрицей

Для j от 0 до N-2 выполнить: // цикл по столбцам

Для i от j+1 до N-1 выполнить: // цикл по строкам ниже диагонали

// зануление элементов R[i][j]

// вычисление коэффициентов вращения $c = \cos(\theta)$, $s = -\sin(\theta)$

$r = \sqrt{(R[j][j]^2 + R[i][j]^2)}$

$c = R[j][j] / r$

$s = -R[i][j] / r$

// вращение двух строк i и j

Для k от 0 до N-1 выполнить (параллельно):

temp_R = R[j][k] * c - R[i][k] * s

R[i][k] = R[j][k] * s + R[i][k] * c

R[j][k] = temp_R

Конец цикла по k

// вращение двух столбцов i и j

Для k от 0 до N-1 выполнить (параллельно):

temp_Q = c * Q[k][j] - s * Q[k][i]

Q[k][i] = s * Q[k][j] + c * Q[k][i]

Q[k][j] = temp_Q

Конец цикла по k

Конец цикла по i

Конец цикла по j

Конец процедуры.

Можно сохранить параметры вращения $\cos\theta$ и $\sin\theta$ и позже, после накопления матрицы R , восстановить матрицы вращения и вычислить матрицу Q .

Для этого не потребуется лишней памяти, параметры вращения $\cos\theta$ и $\sin\theta$ можно сохранить в позиции R_{ij} , брать в цикле для восстановления матриц вращения и вычисления Q , занулять элемент R_{ij} . Для этого $\cos\theta$ и $\sin\theta$ по формуле превратим в одно число:

$$\tau = \frac{s}{1+c}.$$

Обратная его трансформация в $\cos\theta$ и $\sin\theta$ выполняется по формулам:

$$c = \frac{1-\tau^2}{1+\tau^2}, s = \frac{2 \cdot \tau}{1+\tau^2}.$$

Кроме того, хранение в памяти матрицы Q по столбцам с точки зрения устройства компьютерной памяти совсем неудачно — элементы разделены, не будет оптимизации по попаданию нескольких подряд лежащих элементов в кэш. Если хранить матрицу Q транспонированной, обращение будет к элементам одной строки, которые в памяти будут лежать рядом. Должно смениться обращение:

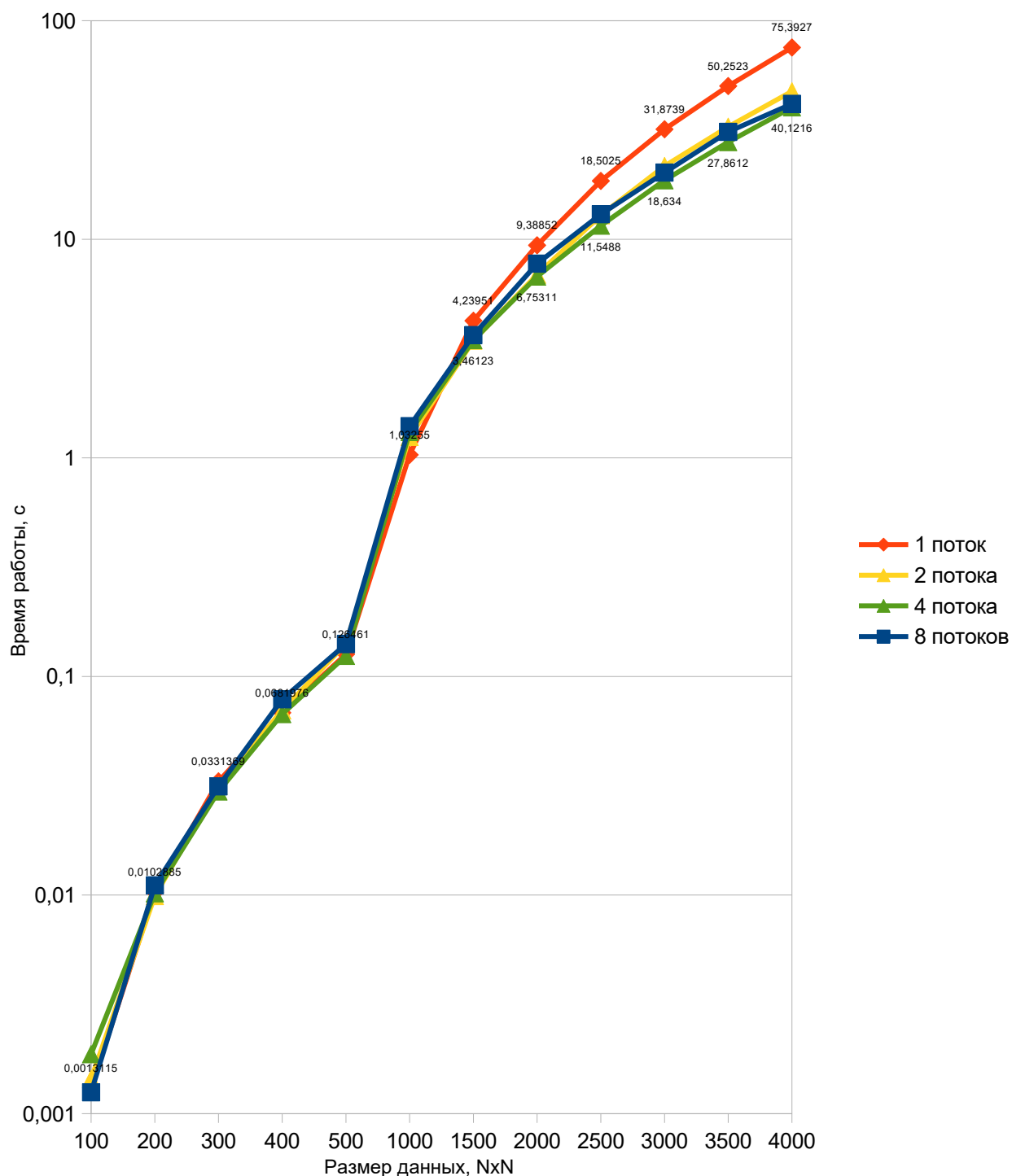
$$\begin{aligned} Q[k][i] &\rightarrow Q[i][k], \\ Q[k][j] &\rightarrow Q[j][k]. \end{aligned}$$

А также в конце выполнения разложения необходимо сделать транспонирование:

Эксперимент показал, что наибольший выигрыш по времени для матрицы A больше 1000×1000 элементов даёт способ, согласно которому нужно вычислять матрицу Q отдельно от матрицы R и хранить Q транспонированной.

Замеры для типа данных `double` (вещественное число двойной точности) на разном количестве потоков дали следующие результаты:

Схема 5 – Замеры времени работы программы на разных размерах матрицы A



Полученное ускорение в виде отношения времени работы программы, запущенной в параллельном режиме, ко времени работы последовательной версии для различного количества потоков можно посмотреть в таблице ниже:

Таблица 5 – Данные об ускорении для первой реализации
QR-разложения с помощью вращений Гивенса в зависимости от количества потоков

Размер данных, NхN	Время работы последовательной версии	Ускорение относительно последовательной версии		
		2 потока	4 потока	8 потоков
100	0,0013115	0,91	0,7	1,05
200	0,0102885	1,04	1,01	0,93
300	0,0331369	1,05	1,11	1,05
400	0,0681976	0,97	1,01	0,87
500	0,126461	0,9	1,02	0,9
1000	1,03255	0,83	0,79	0,74
1500	4,23951	1,23	1,22	1,16
2000	9,38852	1,36	1,39	1,21
2500	18,5025	1,44	1,6	1,42
3000	31,8739	1,47	1,71	1,58
3500	50,2523	1,54	1,8	1,62
4000	75,3927	1,59	1,88	1,81
СРЕДНЕЕ ПОЛУЧЕННОЕ УСКОРЕНИЕ	-	1,19	1,27	1,2

Первая версия QR-разложения по методу Гивенса показала плохую способность к распараллеливанию. Вероятно это из-за того, что области с использованием параллелизма небольшие, пороги для распараллеливания для $N < 1000$ отсутствуют, а главные операции — вращение строк на каждой итерации, не поддаются распараллеливанию в виду последовательной зависимости по данным (нельзя вращать несколько строк одновременно). Кроме того имеет значение порядок зануления элементов: слева направо и сверху вниз. Не получается исполнять вращение независимых строк так, чтобы получить итоговые унитарную матрицу Q и верхнетреугольную матрицу R . Операции вращения хоть и несложные арифметически, но требуют множественных обращений к памяти. Стоит также отметить, что доступ к памяти для матриц был бы эффективнее, если бы они хранилась транспонированно.

Абсолютная ошибка для типа числа с плавающей точкой двойной точности вне зависимости от размера данных имеет порядок $1e-12$, относительная — $1e-15$.

Реализация прикреплена в Приложении 4.

2.2.2 Хранение матриц Q и R в одной структуре

Если использовать подход, при котором матрица Q вычисляется там же, где и матрица R, во время основного цикла вращения, возможно использование специализированной структуры данных с быстрым доступ к элементам обеих матриц. Эта идея берёт за основу тот факт, что алгоритм обращается к i и j строкам матрицы R и одновременно к i и j столбцам матрицы Q, и если хранить матрицу Q транспонированной, произойдёт обращение к одним и тем же элементам R_{ik} и Q^T_{ik} , R_{jk} и Q^T_{jk} во время вращения строк (транспонированных столбцов, в случае матрицы Q).

Небольшой выигрыш по ускорению может дать совместное попарное хранение элементов Q и R благодаря оптимизации доступа к памяти. При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи.

Для уменьшения времени работы программы можно попробовать использовать структуру данных, которая обеспечит лучшее обращение к памяти. Алгоритм использует специализированную структуру данных QRMatrix, которая объединяет хранение матриц Q и R для оптимизации доступа к памяти:

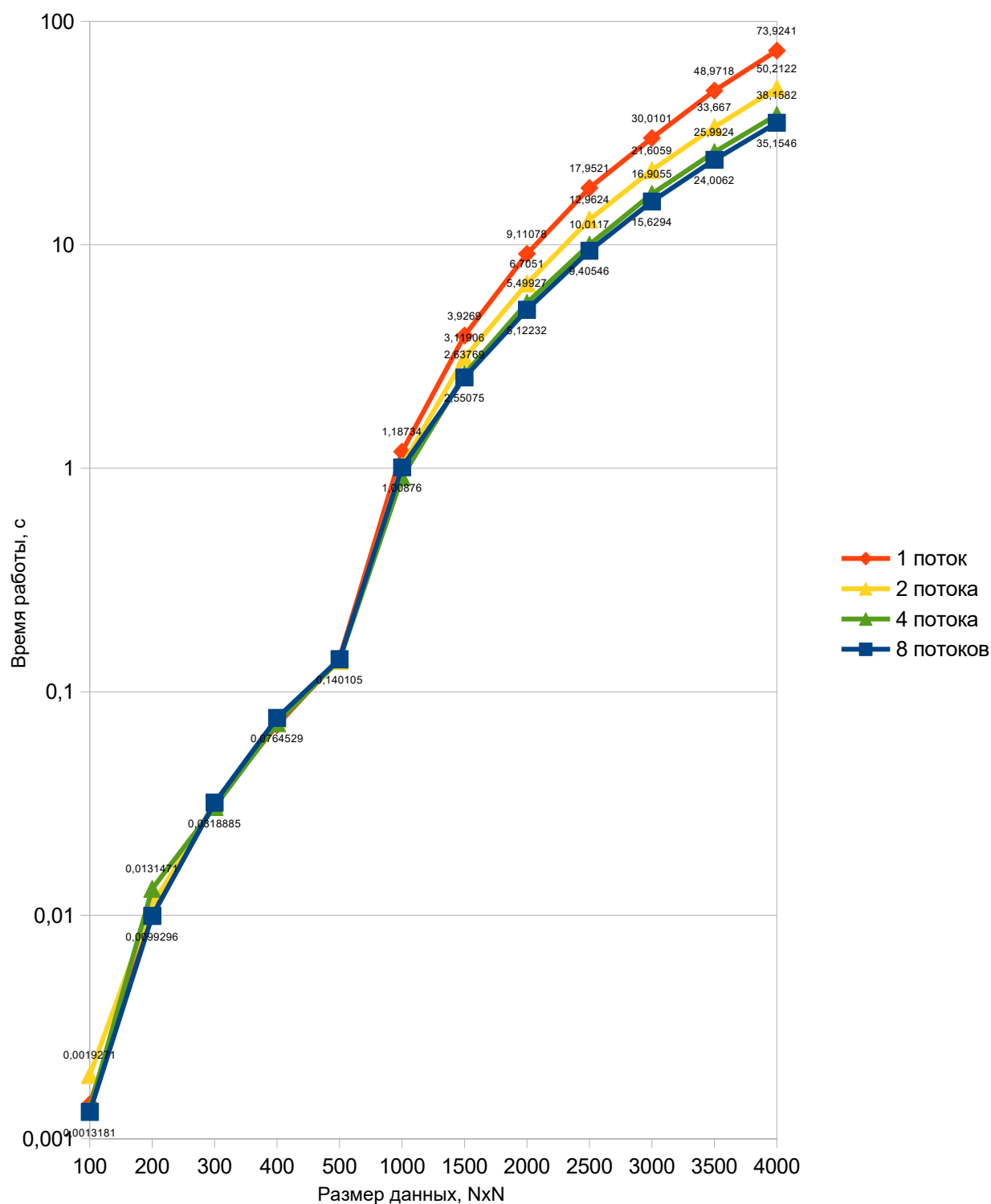
При этом матрица R хранится по строкам, а матрица Q, транспонированная, по столбцам.

Применение вращения затрагивает только две строки матрицы R и два столбца матрицы Q. Для матрицы R обновляются элементы с индексами $k \geq j$, тогда как для матрицы Q преобразуются все элементы соответствующих столбцов. Реализация включает несколько уровней проверок численной устойчивости: игнорируются вращения для пренебрежимо малых элементов и контролируется относительная величина элементов для предотвращения потери точности.

Небольшой выигрыш по ускорению может дать совместное попарное хранение элементов Q и R благодаря оптимизации доступа к памяти. При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи. Это и совместное обновление матриц в одном цикле может дать хорошее ускорение для этой реализации.

Замеры для типа данных double (вещественное число двойной точности) на разном количестве потоков дали следующие результаты:

Схема 6 – Замеры времени работы программы на разных размерах матрицы А



Полученное ускорение в виде отношения времени работы программы, запущенной в параллельном режиме, ко времени работы последовательной версии для различного количества потоков можно посмотреть в таблице ниже:

Таблица 6 – Данные об ускорении для второй реализации QR-разложения с помощью вращений Гивенса в зависимости от количества потоков

Размер данных, NxN	Время работы последовательной версии	Ускорение относительно последовательной версии		
		2 потока	4 потока	8 потоков
100	0,0014271	0,74	1,04	1,08
200	0,0100838	0,9	0,77	1,02
300	0,0319887	1,02	1,05	1
400	0,0708905	0,94	0,98	0,93
500	0,141176	1,03	1,01	1,01
1000	1,18734	1,16	1,3	1,18
1500	3,9269	1,26	1,49	1,54
2000	9,11078	1,36	1,66	1,78
2500	17,9521	1,38	1,79	1,91
3000	30,0101	1,39	1,78	1,92
3500	48,9718	1,45	1,88	2,04
4000	73,9241	1,47	1,94	2,1
СРЕДНЕЕ ПОЛУЧЕННОЕ УСКОРЕНИЕ	-	1,18	1,39	1,46

Вторая версия метода Гивенса немного лучше первой, но параллелизация всё ещё неэффективна. Всё ещё дают о себе знать промахи кэша и плохая локальность. Структура данных, хранящая в себе элементы Q и R рядом, не оправдала себя. Следует использовать другой способ оптимизации доступа к памяти. Большое количество потоков только создало дополнительные накладные расходы на создание и синхронизацию по сравнению с теми небольшими объёмами вычислений, для которых они были созданы.

Абсолютная ошибка для типа числа с плавающей точкой двойной точности вне зависимости от размера данных имеет порядок $1e-12$, относительная — $1e-15$.

Реализация прикреплена в Приложении 5.

2.2.3 Использование SIMD

При использовании отдельных контейнеров для Q и R в нынешней реализации с вложенными циклами происходит нарушение локальности данных, при параллельном доступе к Q и R возникают множественные кэш-промахи.

Для максимального эффекта по части улучшения обращения к памяти можно использовать векторизацию и SIMD.

Векторизация — вид распараллеливания программного кода, при котором однопоточные программы, исполняющие одну операцию над одним элементом (single instruction-single data) в каждый момент времени, модифицируются таким образом, чтобы они могли выполнять более одной операции одновременно. Скалярные операции, обрабатывающие пару аргументов заменяются на векторные аналоги, способные производить одну и ту же операцию над массивом данных в единицу времени. Векторная обработка данных широко применяется как в лабораторных суперкомпьютерах, так и в обычных домашних компьютерах.

SIMD (Single Instruction, Multiple Data) — это технология из области компьютерных высокоэффективных вычислений, позволяющая с помощью одной машинной инструкции (single instruction) процессора обрабатывать несколько элементов данных (multiple data). SIMD-инструкции используют специальные векторные регистры, в которые помещается несколько элементов. Чтение/запись, арифметические операции над этими регистрами выполняются процессором одновременно. Количество данных, которые могут храниться и обрабатываться одновременно, зависит от размеров регистра и данных. Некоторые распространённые SIMD-расширения современных процессоров:

- SSE (Streaming SIMD Extensions) — 128-битные регистры, позволяющие обрабатывать четыре 32-битных значения с плавающей запятой одинарной точности в один момент времени. Расширение SSE2 позволило использовать два 64-битных значения с плавающей точкой двойной точности, или два 64-битных целых числа, или четыре 32-битных, или восемь 16-битных коротких целых чисел, или шестнадцать 8-битных байтов или символов.
- AVX (Advanced Vector Extensions) — 256-битные регистры, поддерживающие более сложные операции. В архитектуре x86-64 AVX поддерживает шестнадцать 256-битных регистров YMM. Каждый регистр YMM может хранить и выполнять одновременные операции над:
 - восемью 32-битными числами с плавающей запятой одинарной точности;
 - четырьмя 64-битными числами с плавающей запятой двойной точности.

- AVX-512 — расширение с увеличенным размером векторных регистров, которое увеличивает размер каждого регистра до 512 бит, что, очевидно, позволяет обрабатывать больше данных за одну операцию.

Автоматическую векторизацию некоторым образом обеспечивают компиляторы. Современные представители, GCC, Clang, MSVC, могут анализировать код и заменять скалярные инструкции на векторные с использованием SIMD-инструкций.

Также компилятору с помощью директив можно указать принудительное использование simd-инструкций. `#pragma simd` для компиляторов Intel C/C++ позволяет указать, что компилятор должен явно векторизовать цикл, а `#pragma omp simd` — директива для той же цели из стандарта OpenMP.

Кроме того, можно использовать так называемые `intrinsic`'и, встроенные функции для работы с SIMD-инструкциями. Они позволяют использовать команды расширений с синтаксисом, похожим на вызов C-подобных функций, при этом компилятор сам выполнит оптимизацию кода.

Для векторизации кода и использования `simd` матрицы `Q` и `R` будем хранить в ленточном виде, записав в одномерный массив матрицы по строкам (матрица `Q` снова будет храниться транспонированной).

Ожидается, что SIMD-версия даст значительный прирост к производительности алгоритма, уменьшив время исполнения программы, так как она обеспечивает лучшее использование аппаратных возможностей современных процессоров.

Для векторных вычислений можно использовать встроенную в OpenMP векторизацию или использовать AVX-интринсики.

Использование `#pragma omp simd` перед циклом даст явно знать компилятору, что этот цикл может быть векторизован. Директива указывает, что итерации цикла независимы и могут быть выполнены параллельно с использованием векторных инструкций процессора. Циклы вращения строк матриц `Q` и `R` из базовой версии могут без изменения быть использованы в SIMD-версии.

Для `#pragma omp simd` есть клауза `aligned`, помечающая, что память, над которой производятся вычисления, выровнена по границе размера типа (64-кратные адреса для 64-битных чисел с плавающей точкой двойной точности и 32-кратные адреса для 32-битных чисел с плавающей точкой одинарной точности). Для ещё более лучшего обращения к памяти следует выравнивать строки матриц `Q` и `R` по границам 64 или 32, в зависимости от типа.

Векторную обработку также можно добавить с помощью интринсиков, используя специальные функции, которые самостоятельно производят работу с памятью и регистрами. Идея их использования та же: при вращении строк матриц Q и R , производя одновременно вычисления четырёх (при работе с вещественными числами двойной точности) или восьми (при работе с вещественными числами одинарной точности) значений в двух строках одновременно.

`__m256d` (256 бит для хранения 8-байтных вещественных чисел двойной точности) и `__m256` (256 бит для хранения 4-байтных вещественных чисел одинарной точности) — это типы данных, позволяющие работать с AVX-интринсиками. Функции с префиксом `_mm256_*` позволяют загружать и читать данные из регистра, производить над ними математические операции сложения, умножения и др.

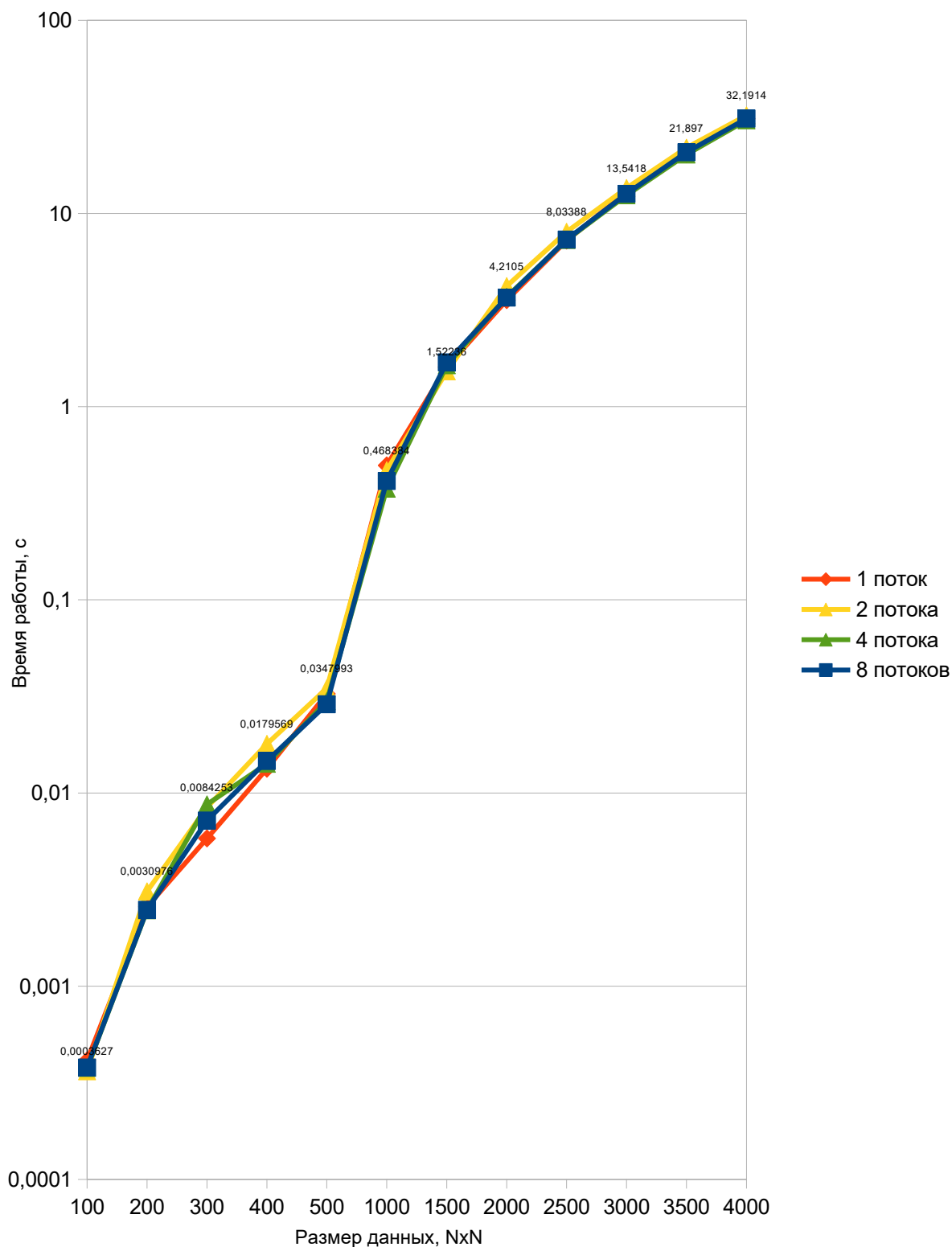
Эта операция выполняется одновременно для столько чисел, сколько помещается в 256-битном регистре, оставшийся «хвост» обрабатывается старым способом, без векторизации.

Точно тот же подход выполняется для восстановления с помощью матриц вращения матрицы Q .

К сожалению, мне не удалось заставить мой компилятор использовать `#pragma omp simd` с клаузой `aligned`, при компиляции появлялось предупреждение о том, что она игнорируется. Тем не менее, использование `simd` от OpenMP показало лучший результат по сравнению с предыдущими реализациями QR разложения на основе вращений Гивенса. Однако незначительный разрыв по времени есть, между использованием интринсиков и `simd` прагмы есть небольшая разница. Лучшее время всё-таки показала реализация с интринсиками.

Замеры для типа данных `double` (вещественное число двойной точности) на разном количестве потоков дали следующие результаты:

Схема 7 – Замеры времени работы программы на разных размерах матрицы A



Полученное ускорение в виде отношения времени работы программы, запущенной в параллельном режиме, ко времени работы последовательной версии для различного количества потоков можно посмотреть в таблице ниже:

Таблица 7 – Данные об ускорении для первой реализации
QR-разложения с помощью вращений Гивенса в зависимости от количества потоков

Размер данных, NхN	Время работы последовательной версии	Ускорение относительно последовательной версии		
		2 потока	4 потока	8 потоков
100	0,0004123	1,14	1,09	1,09
200	0,00258	0,83	1,04	1,04
300	0,0058212	0,69	0,67	0,81
400	0,0133668	0,74	0,94	0,91
500	0,0326057	0,94	1,09	1,13
1000	0,496281	1,06	1,32	1,2
1500	1,60407	1,05	0,98	0,95
2000	3,55704	0,84	0,96	0,97
2500	7,23234	0,9	0,99	0,99
3000	12,8809	0,95	1,04	1,02
3500	20,2517	0,92	1	0,98
4000	30,9537	0,96	1,02	1
СРЕДНЕЕ ПОЛУЧЕННОЕ УСКОРЕНИЕ	-	0,92	1,01	1,01

При использовании SIMD-векторизации добавление многопоточности не дало преимущества и даже ухудшило производительность. Вероятно это из-за того, что потокам нужно хранить в кэше для корректного исполнения SIMD инструкций разные данные, происходят постоянные конфликты и промахи. Зато последовательная версия показала себя хорошо, видно, что третья реализация метода Гивенса даже при последовательном выполнении отрабатывает за лучшее, меньшее время по сравнению со второй реализацией вращений Гивенса. Для небольших данных (матриц 500х500 и меньше) этот алгоритм превосходит даже оптимизированный алгоритм на основе отражений Хаусхолдера.

Абсолютная ошибка для типа числа с плавающей точкой двойной точности вне зависимости от размера данных имеет порядок $1e-12$, относительная — $1e-15$.

Реализация прикреплена в Приложении 6.

Сводные данные ниже содержат сравнение всех полученных реализаций QR-разложения на основе вращений Гивенса с первой последовательной реализацией. Последним столбцом прикреплено время работы QR-разложения из библиотеки Scilab. Условия исполнения: максимально доступное количество потоков в тестируемой системе (восемь), произвольная плотная матрица A , тип данных матрицы A – double, число с плавающей точкой двойной точности.

Схема 8 – Сравнение версий метода Гивенса с версией из библиотеки Scilab

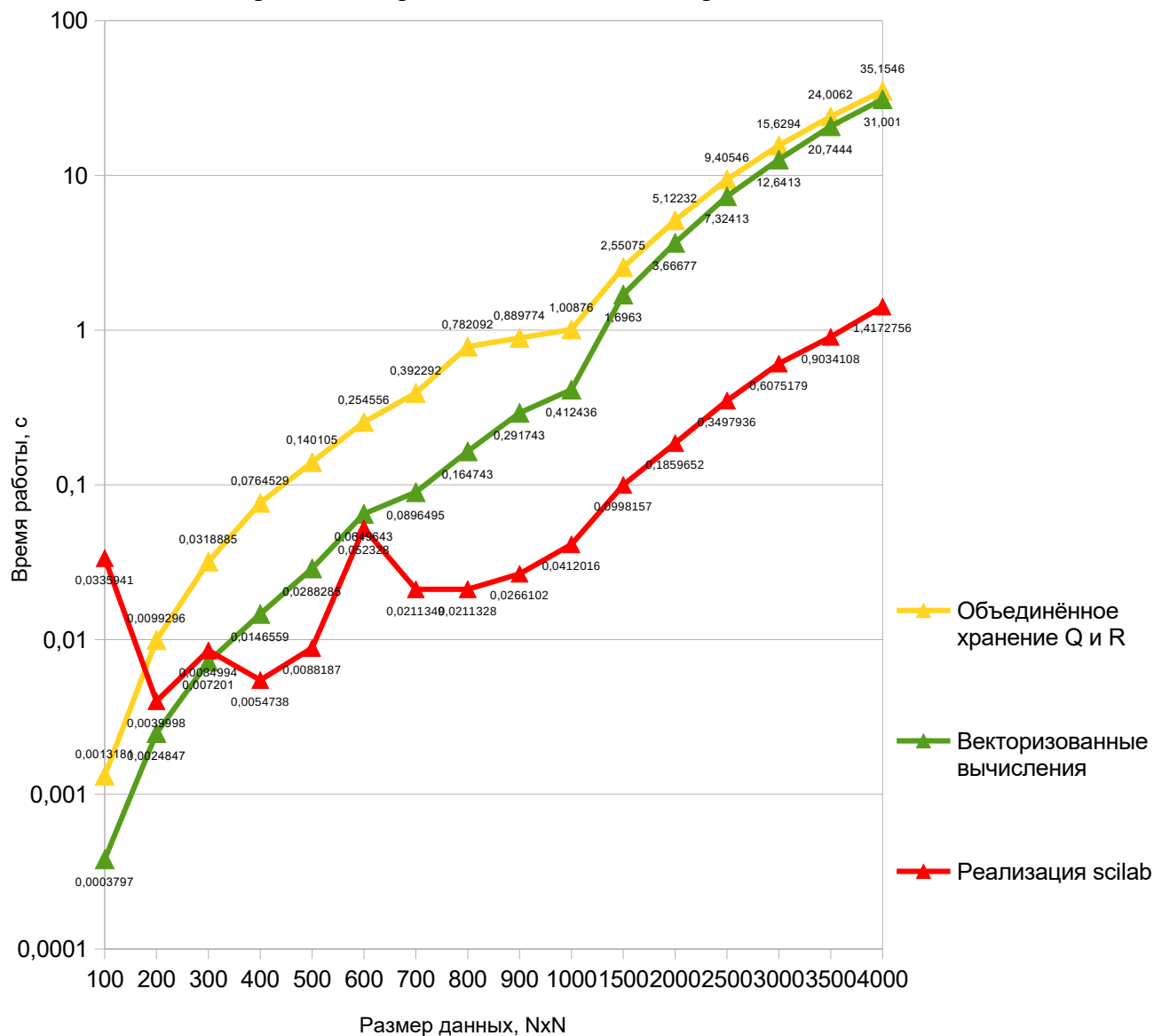


Таблица 8 – Сравнение версий метода Гивенса с версией из библиотеки Scilab

Размер данных (матрица NxN)	Время работы первой последовательной реализации QR- разложения с вращениями Гивенса , с	Время работы QR- разложения с вращениями Гивенса с улучшенным обращением к памяти, с	Время работы QR- разложения с вращениями Гивенса с векторизацией, с	Время работы QR- разложения из библиотеки Scilab, с
100	0,0013115	0,0013181	0,0003797	0,0335941
200	0,0102885	0,0099296	0,0024847	0,0039998
300	0,0331369	0,0318885	0,007201	0,0084994
400	0,0681976	0,0764529	0,0146559	0,0054738
500	0,126461	0,140105	0,0288285	0,0088187
1000	1,03255	1,00876	0,412436	0,0412016
1500	4,23951	2,55075	1,6963	0,0998157
2000	9,38852	5,12232	3,66677	0,1859652
2500	18,5025	9,40546	7,32413	0,3497936
3000	31,8739	15,6294	12,6413	0,6075179
3500	50,2523	24,0062	20,7444	0,9034108
4000	75,3927	35,1546	31,001	1,4172756
СРЕДНЕЕ УСКОРЕНИЕ ПО СРАВНЕНИЮ С ПЕРВОЙ ВЕРСИЕЙ ПО МЕТОДУ ГИВЕНСА	-	1,47	3,22	30,46

Видно, что самый оптимальный и устойчивый результат показывает метод из библиотеки Scilab. В Scilab QR-разложение реализуется с использованием подпрограмм из библиотеки Lapack. Для вещественных матриц используются процедуры DGEQRF, DGEQPF и DORGQR, а для комплексных — ZGEQRF, ZGEQPF и ZORGQR.

Некоторые особенности реализации в Scilab:

- Если матрица X имеет размер $m \times N$ и $m > N$, то вычисляются только первые N столбцов матрицы Q и первые N строк матрицы R (так называемый «экономичный» режим).
- Если столбец матрицы X является линейной комбинацией предыдущих столбцов, то элементы матрицы R после определённого столбца будут равны нулю, и в этом случае R будет верхней трапецевидной.

- Если матрица X имеет ранг rk , то строки $R(rk+1, :)$, $R(rk+2, :)$ и т. д. будут нулевыми.

2.2.4 QR-разложение Хессенберговой матрицы

Верхняя матрица Хессенберга — это квадратная матрица $N \times N$, у которой $a_{ij}=0$, если $i > j+1$.

Нижняя матрица Хессенберга — это квадратная матрица $N \times N$, у которой $a_{ij}=0$, если $j > i+1$.

Матрица, являющаяся одновременно и верхней, и нижней матрицами Хессенберга, трёхдиагональна.

Сводные данные ниже содержат сравнение времени решения задачи разложения верхней хессенберговой матрицы на Q и R для самых оптимальных версий QR-разложения на основе вращений Гивенса и отражений Хаусхолдера, а также версии из библиотеки SciLab. Замеры для типа данных double (вещественное число двойной точности) на максимально доступном количестве потоков в тестируемой системе (восьми):

Схема 9 – Сравнение скорости решения QR-разложения верхней хессенберговой матрицы методом Гивенса, методом Хаусхолдера и QR разложением из библиотеки SciLab

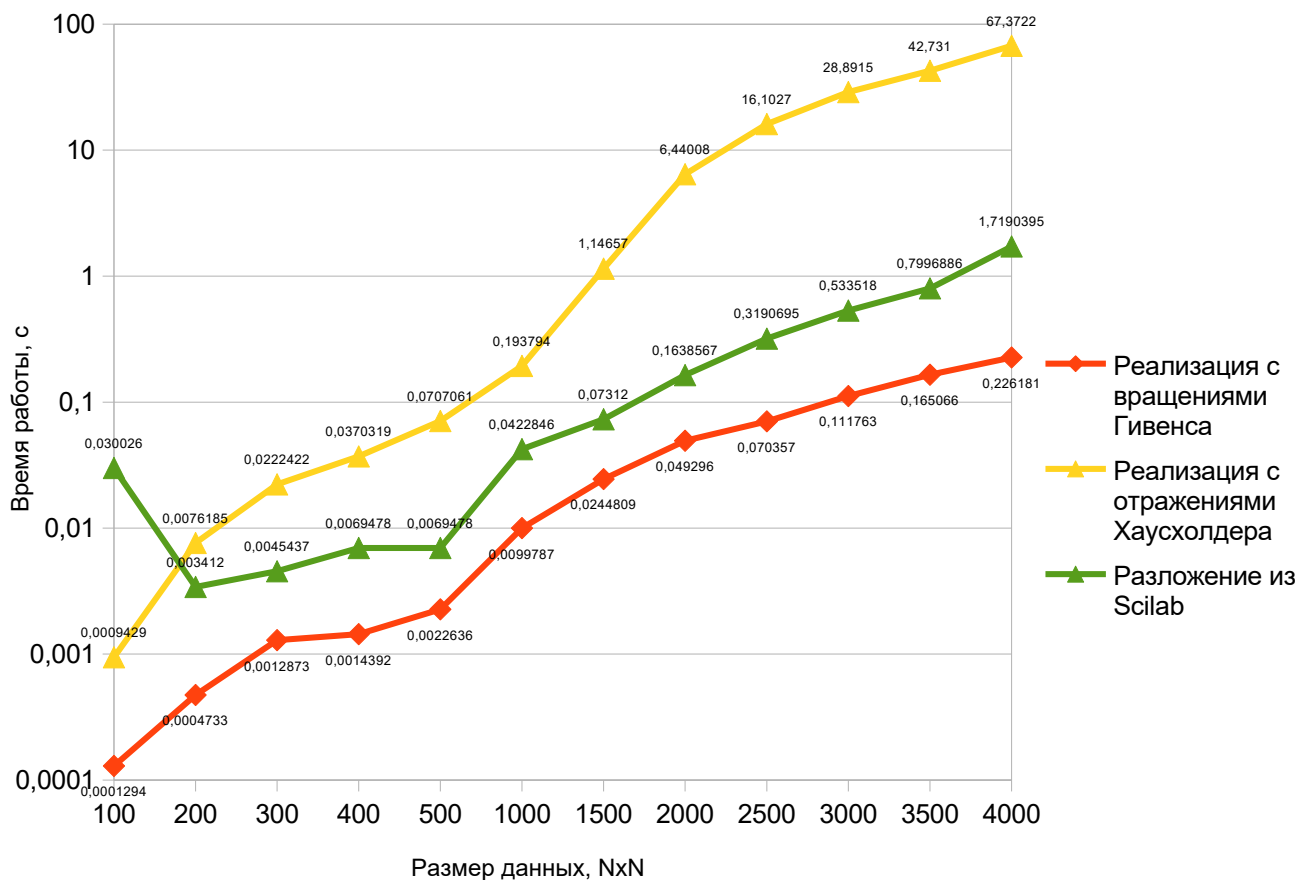


Таблица 9 – Сравнение скорости решения QR-разложения верхней хессенберговой матрицы методом Гивенса, методом Хаусхолдера и QR разложением из библиотеки SciLab

Размер данных (матрица NxN)	Время работы QR- разложения с вращениями Гивенса с векторизацией, с	Время работы QR- разложения на основе отражений Хаусхолдера, с	Время работы QR- разложения из библиотеки SciLab, с
100	0.0001294	0.0009429	0.030026
200	0.0004733	0.0076185	0.003412
300	0.0012873	0.0222422	0.0045437
400	0.0014392	0.0370319	0.0069478
500	0.0022636	0.0707061	0.0107381
1000	0.0099787	0.193794	0.0422846
1500	0.0244809	1.14657	0.07312
2000	0.049296	6.44008	0.1638567
2500	0.070357	16.1027	0.3190695
3000	0.111763	28.8915	0.533518
3500	0.165066	42.731	0.7996886
4000	0.226181	67.3722	1.7190395

Видно, что моя реализация QR-разложения методом вращений Гивенса решила задачу QR-разложения верхней хессенберговой матрицы быстрее, так как, по-видимому, SciLab использует метод отражений Хаусхолдера, который не может игнорировать отдельно стоящие пустые значения в столбцах, а вынужден производить отражение и занулять весь столбец целиком, даже не смотря на то, что в нём всего один значащий элемент.

Сводные данные ниже содержат сравнение времени решения задачи разложения нижней хессенберговой матрицы на Q и R для самых оптимальных версий QR-разложения на основе вращений Гивенса и отражений Хаусхолдера, а также версии из библиотеки SciLab. Замеры для типа данных double (вещественное число двойной точности) на максимально доступном количестве потоков в тестируемой системе (восьми):

Схема 10 – Сравнение скорости решения QR-разложения нижней хессенберговой матрицы методом Гивенса, методом Хаусхолдера и QR разложением из библиотеки SciLab

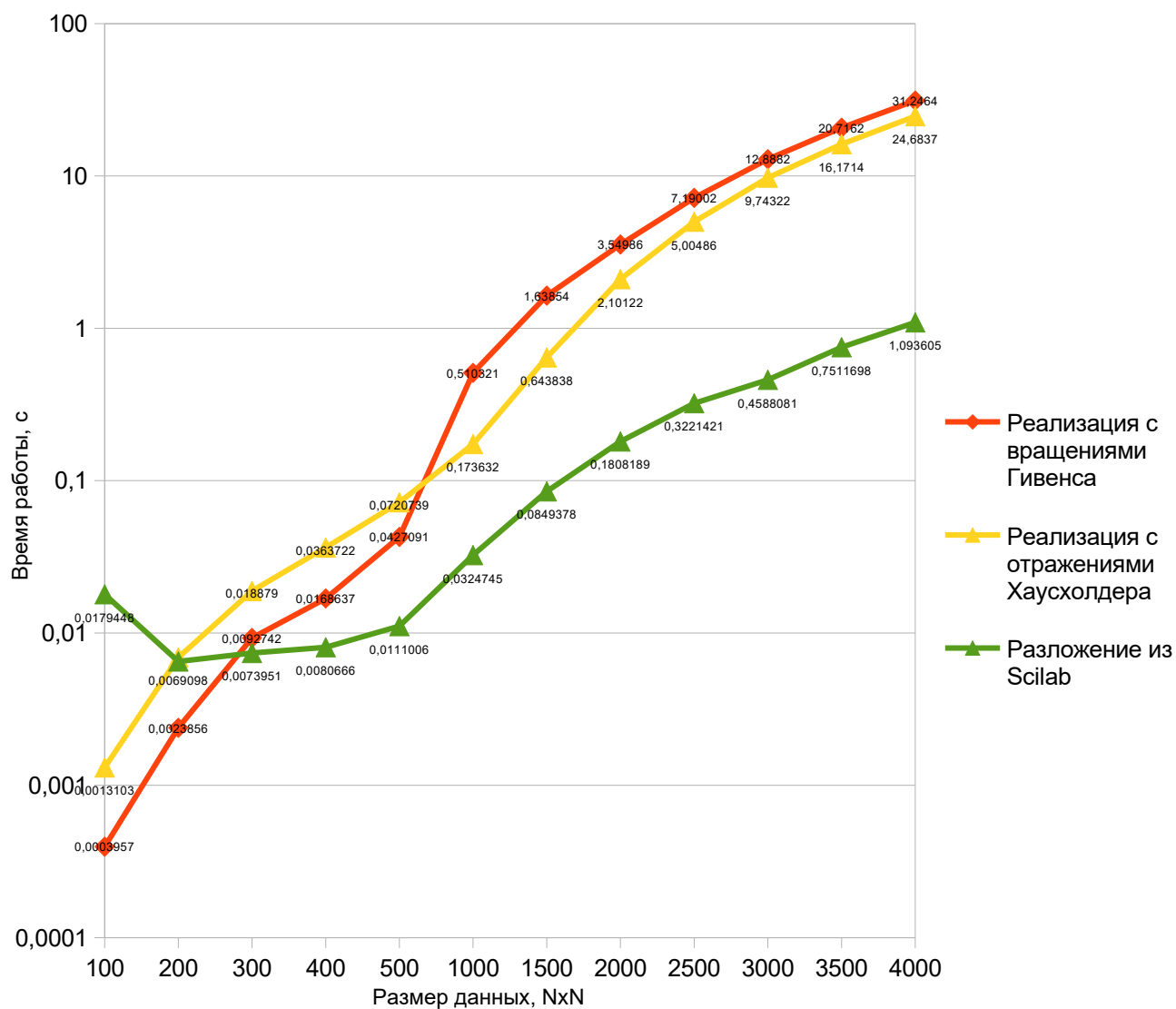


Таблица 10 – Сравнение скорости решения QR-разложения нижней хессенберговой матрицы методом Гивенса, методом Хаусхолдера и QR разложением из библиотеки SciLab

Размер данных (матрица NxN)	Время работы QR- разложения с вращениями Гивенса с векторизацией, с	Время работы QR- разложения на основе отражений Хаусхолдера, с	Время работы QR- разложения из библиотеки SciLab, с
100	0.0003957	0.0013103	0.0179448
200	0.0023856	0.0069098	0.0065019
300	0.0092742	0.018879	0.0073951
400	0.0168637	0.0363722	0.0080666
500	0.0427091	0.0720739	0.0111006
1000	0.510321	0.173632	0.0324745
1500	1.63854	0.643838	0.0849378
2000	3.54986	2.10122	0.1808189
2500	7.19002	5.00486	0.3221421
3000	12.8882	9.74322	0.4588081
3500	20.7162	16.1714	0.7511698
4000	31.2464	24.6837	1.093605

Видно, что моя реализация QR-разложения методом вращений Гивенса справилась с решением задачи QR-разложения нижней хессенберговой матрицы хуже, опять по той же причине, что Scilab использует метод отражений Хаусхолдера, который быстрее зануляет целые столбцы.

Заключение

В заключение хотелось бы подытожить полученные результаты.

Первая версия QR-разложения по методу Хаусхолдера показала плохие результаты за счёт двух громоздких $N \times N$ матричных умножений на каждой из $(N-1)$ итераций, а также из-за явного хранения $N \times N$ матрицы отражения H на каждом шаге. Зато эта версия получило отличное ускорение в 4 раза при увеличении потоков до восьми.

Вторая версия метода Хаусхолдера без матричных умножений очевидно быстрее за счёт устранения ключевого узкого места. Вместо явных умножений реализация использует прямое применение отражений через скалярные произведения. Эта версия распараллеливается хуже первой, прирост производительности в среднем в 2,5 раза на 8 потоках.

Третья версия метода Хаусхолдера с in-place хранением векторов Хаусхолдера оказалась ещё быстрее, за счёт полного исключения отдельного хранилища для векторов Хаусхолдера v_k , $k \in [0, N-1]$ и максимального использования in-place вычислений, что сократило число обращений к памяти и улучшило локальность данных при последовательном доступе как для сохранения, так и для извлечения векторов отражений. Эта версия также при увеличении числа потоков до восьми получила прирост производительности в среднем только в 2,5 раза.

Первая версия метода вращений Гивенса благодаря своей простоте показала неплохую производительность, лишь на больших данных и показала почти замедление при увеличении количества потоков. Вероятно, это произошло в виду следующих причин:

- Слишком маленькая работа внутри параллельных секций не дала ожидаемого прироста;
- Отсутствие порога параллелизации для малых N : увеличение накладных расходов не даёт прироста на данных меньше 1000×1000 элементов;
- Row-major доступ к матрицам и плохая локальность.

Вторая версия не сильно превосходит первую для малых N , существенно обходя её по эффективности лишь для данных более 2000×2000 элементов.

Видно, что третья реализация метода Гивенса даже выполняясь последовательно отрабатывает за лучшее, меньшее время по сравнению со второй реализацией QR-разложения на основе вращений Гивенса. Для небольших данных (матриц 500×500 и меньше) этот алгоритм превосходит даже оптимизированный алгоритм на основе отражений

Хаусхолдера. Однако на данных больше 1000×1000 элементов всё ещё лучшим показывает себя QR-разложение на основе отражений Хаусхолдера.

Не смотря на оптимизацию хранения данных, можно заметить, что реализация метода Гивенса существенно проигрывает реализации по методу Хаусхолдера в скорости из-за необходимости последовательной обработки всех поддиагональных элементов, тогда как Хаусхолдер обнуляет целый столбец за одно преобразование. Даже при использовании единого контейнера QRMatrix и единого цикла метод не даёт на плотных матрицах более значительных результатов.

Различные версии алгоритма QR-разложения на основе вращений Гивенса в целом работают быстрее чем, QR-разложение на основе отражений Хаусхолдера, на небольших данных ($< 1000 \times 1000$ элементов), однако всё ещё проигрывает по скорости на больших ($>> 1000 \times 1000$ элементов).

Дело в ограниченной применимости метода. В виду его особенности обнуления отдельных элементов вместо целых столбцов под диагональю, как в методе Хаусхолдера, этот метод даст лучший результат при применении его к разреженным матрицам. Игнорирование нулевых отдельно стоящих значений позволит выполнить QR разложение быстрее. Так работают, например, открытые известные реализации: при работе с плотными матрицами используется метод Хаусхолдера, а при работе с разреженными матрицами — метод Гивенса.

Также можно попробовать реализовать блочный алгоритм для дальнейшего нивелирования эффекта от обращений к памяти во время выполнения вращений Гивенса. Потребуется использовать согласованную общую память, разделяемую отдельными потоками, отвечающими за разные блоки. Для этого больше подойдёт MPI-параллелизм на уровне процессов. Это позволит приблизиться к версии из библиотеки Scilab, ведь, как уже было сказано выше, в Scilab QR-разложение реализуется с использованием подпрограмм из библиотеки Lapack — библиотеку, написанную на Fortran с:

- Блочными алгоритмами (более оптимальное использование кэша),
- Высокооптимизированными многопоточными BLAS — Basic Linear Algebra Subprograms (Intel MKL/OpenBLAS),
- Ручной векторизацией и ассемблерными вставками.

Полный код работы можно посмотреть по ссылке <https://github.com/wellcome2AD/QR-decomposition>.

Список литературы

1. Horn, R. A., Johnson C. R. Matrix analysis: Textbook / Roger A. Horn, Charles R. Johnson. – 2nd ed. — Cambridge University Press, 2012. — 561 p. — URL: <https://doi.org/10.1017/CBO9780511810817> (дата обращения: 14.12.2024).
2. Белов, С.А., Золотых Н.Ю. Численные методы линейной алгебры. Лабораторный практикум. / С.А. Белов, Н.Ю. Золотых — Нижний Новгород: Изд-во Нижегородского государственного университета им. Н.И. Лобачевского, 2005. — 264 с.
3. Agullo, E. Exploiting a Parametrized Task Graph model for the parallelization of a sparse direct multifrontal solver // Euro-Par 2016: Parallel Processing Workshops, 2017. – P. 175–186. – URL: https://doi.org/10.1007/978-3-319-58943-5_14 (дата обращения: 14.12.2024).
4. Davis, T. A. Algorithm 915, SuiteSparseQR: Multifrontal multithreaded rank-revealing sparse QR factorization / T. A. Davis // ACM Transactions on Mathematical Software, Vol. 38, No. 1, Article 8, 2011. – P. 1-22. – URL: <https://doi.org/10.1145/2049662.2049670> (дата обращения: 14.12.2024).
5. Новак, А. Е. Параллельный алгоритм построения QR-разложения разреженных матриц для систем с общей памятью / А. Е. Новак, С. А. Лебедев // Суперкомпьютерные дни в России : Труды международной конференции, Москва, 24–25 сентября 2018 года / Суперкомпьютерный консорциум университетов России, Российская академия наук. – Москва: Московский государственный университет имени М.В. Ломоносова" Издательский Дом (типография), 2018. – С. 841-847. – URL: <https://2018.russianscdays.org/files/pdf18/841.pdf> (дата обращения: 14.12.2025).
6. Soliverez, C. E. Orthonormalization on the plane: a geometric approach / C. E. Soliverez, E. Gagliano // Revista Mexicana de Física., 1985. – P. 743–758. – URL: https://www.researchgate.net/publication/267471614_Orthonormalization_on_the_plane_a_geometric_approach (дата обращения: 14.12.2024).
7. Вержбицкий, В. М. Численные методы. Линейная алгебра и нелинейные уравнения : Учебник / В. М. Вержбицкий. – Москва : Высшая школа, 2005. – 432 с.
8. Фролов, А. В. Сопоставление разных методов решения одной задачи по методике проекта Algowiki / А. В. Фролов, А. С. Антонов, Вл. В. Воеводин, А. М. Теплов // Параллельные вычислительные технологии (ПаВТ'2016), 2016 – С. 347–360. – URL: <https://ceur-ws.org/Vol-1576/062.pdf> (дата обращения: 14.12.2024).

Приложение 1. QR-разложение с использованием отражений Хаусхолдера, первая реализация

```
// QR-decomposition\src\HouseholderMethod\HouseholderBasic.h
#pragma once

#include <vector>

#include "../IQRSolver.h"
#include "Matrix/MatrixOperations.h"

template <typename T>
class HouseholderMethodBasic : public IQRSolver<T> {
    // первая, самая неоптимальная версия. на каждом шаге алгоритма выполняется два
    // матричных умножения,
    // используется много лишней памяти
public:
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
std::vector<std::vector<T>>& Q, std::vector<std::vector<T>>& R) override
    {
        const ptrdiff_t N = A.size();
        R = A;
        Q = std::vector<std::vector<T>>(N, std::vector<T>(N, 0));
        for (int i = 0; i < N; ++i) {
            Q[i][i] = 1.0;
        }

        for (ptrdiff_t k = 0; k < N - 1; ++k) {
            // count beta
            double sum = 0.0;
#pragma omp parallel for num_threads(thread_num) reduction(+:sum)
            for (ptrdiff_t i = k; i < N; ++i) {
                sum += R[i][k] * R[i][k];
            }
            double beta = sign(-R[k][k]) * sqrt(sum);

            // count mu
            double mu = 1.0 / sqrt(2.0 * beta * (beta - R[k][k]));

            // count v
            std::vector<double> v(N, 0);
#pragma omp parallel for num_threads(thread_num)
            for (int i = k; i < N; ++i) {
                v[i] = R[i][k] * mu;
            }
            v[k] -= beta * mu;

            // count H
            std::vector<std::vector<T>> H(N, std::vector<T>(N, 0));
#pragma omp parallel for num_threads(thread_num)
            for (int i = 0; i < N; ++i) {
                H[i][i] = 1.0;
                for (int j = 0; j < N; ++j) {
                    H[i][j] -= v[i] * v[j] * 2.0;
                }
            }

            R = multiplyMatrix(H, R);
            Q = multiplyMatrix(Q, H);
        }
    }
}
```



```

private:
    static double sign(double x)
    {
        if (x > 0.0)
            return 1.0;
        if (x < 0.0)
            return -1.0;
        return 1.0;
    }
};

// QR-decomposition\src\Matrix\MatrixOperations.h
#pragma once

#include <iomanip>
#include <vector>
#include <string>
#include <cassert>
#include <random>
#include <algorithm>

/*остальная часть кода библиотеки*/

template <typename T>
inline std::vector<std::vector<T>> multiplyMatrix(const std::vector<std::vector<T>>&
m1, const std::vector<std::vector<T>>& m2, int block_size=50) {
    assert(m1.size() == m2[0].size());
    auto N = m1.size();
    std::vector<std::vector<T>> res(N, std::vector<T>(N, 0));
#pragma omp parallel for num_threads(thread_num)
    for (int i = 0; i < N; i++) {
        for (int jj = 0; jj < N; jj += block_size) {
            for (int kk = 0; kk < N; kk += block_size) {
                for (int j = jj; j < ((jj + block_size) > N ? N : (jj +
block_size)); j++) {
                    double temp = 0.0;
                    for (int k = kk; k < ((kk + block_size) > N ? N : (kk
+ block_size)); k++) {
                        temp += static_cast<double>(m1[i][k]) *
static_cast<double>(m2[k][j]);
                    }
                    res[i][j] += static_cast<T>(temp);
                }
            }
        }
    }
    return res;
}

```

Приложение 2. QR-разложение с использованием отражений Хаусхолдера, реализация без матричных умножений

```
#pragma once

#include <vector>

#include "../IQRSolver.h"

template <typename T>
class HouseholderMethodWithoutMatrixMults : public IQRSolver<T> {
public:
    // вторая версия, избавлена от недостатка первой -- двух матричных умножений на
    // каждой итерации
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
std::vector<std::vector<T>>& Q, std::vector<std::vector<T>>& R) override
    {
        const size_t N = A.size();
        // Конвертируем входные данные в плоские массивы
        std::vector<T> A_flat(N * N), Q_flat;
        for (ptrdiff_t j = 0; j < N; ++j) {
            for (ptrdiff_t i = 0; i < N; ++i) {
                A_flat[i * N + j] = A[i][j];
            }
        }

        auto R_flat = A_flat;
        std::vector<T> all_v(N * (N - 1), 0.0); // все вектора Хаусхолдера будут
хранятся в одной матрице подряд, без незначащих нулей

        // вычисление R
        //  $H * R = (I - 2 * v * v^T) * R = R - 2 * v * (v^T * R)$ 
        for (ptrdiff_t k = 0; k < N - 1; ++k)
        {
            T* v_k = &all_v[k * N]; // часть матрицы с вектором vk
            ptrdiff_t v_length = N - k; // длина вектора vk
            ptrdiff_t kN = k * N; // предвычисленное смещение для k строки

            T* R_col_k = &R_flat[kN + k]; // 0 элемент k столбца матрицы R,
чтобы получить следующий -- прибавить строку

            double beta = 0.0;
            // count beta
            double sum = 0.0;
#pragma omp parallel for num_threads(thread_num) reduction(+:sum)
            for (int i = 0; i < v_length; ++i) {
                sum += R_col_k[i * N] * R_col_k[i * N];
            }
            if (fabs(sum) < std::numeric_limits<T>::epsilon()) // если
знаменатель близок к 0, вектор отражения не нужен
            {
                continue;
            }
            double norm_x = sqrt(sum);
            beta = (R_flat[kN + k] >= 0.0) ? -norm_x : norm_x;

            double mu = 0.0;
            // count mu
            double denominator = 2.0 * beta * (beta - R_flat[kN + k]);
            if (fabs(denominator) < std::numeric_limits<T>::epsilon()) // если
знаменатель близок к 0, вектор отражения не нужен
            {
                continue;
            }
        }
    }
};
```

```

    }
    mu = 1.0 / sqrt(denominator);

    // count v
#pragma omp parallel for num_threads(thread_num)
    for (int i = 0; i < v_length; ++i) {
        v_k[i] = R_col_k[i * N] * mu;
    }
    v_k[0] -= beta * mu;

    // отражение
    //  $R - 2 * v * (v^T * R)$ 
    std::vector<double> _2vTR(N - k); // предварительно вычисляем  $v^T R$ 
для каждого столбца
#pragma omp parallel for num_threads(thread_num)
    for (ptrdiff_t j = k; j < N; ++j) {
        T* R_col_j = &R_flat[kN + j]; // начальный элемент для
умножения из j-ого столбца R, прибавить строку
        double vTR = 0.0;
        for (int i = 0; i < v_length; ++i) {
            vTR += v_k[i] * R_col_j[i * N];
        }
        _2vTR[j - k] = 2.0 * vTR;
    }

    // обновление R
    //  $R - v * 2v^T R$ 
#pragma omp parallel for num_threads(thread_num)
    for (ptrdiff_t j = k; j < N; ++j) {
        for (int i = 0; i < v_length; ++i) {
            T* R_col_j = &R_flat[kN + j];
            R_col_j[i * N] -= v_k[i] * _2vTR[j - k];
        }
    }
}

// вычисление Q
// начать с  $Q_{\{N-1\}} = H_{\{N-2\}} * I = I - 2 * v_{\{N-2\}} * (v_{\{N-2\}}^T * I)$  и
домножать слева на матрицы отражения
//  $Q = H_{\{1\}} * \dots * (H_{\{k\}} * Q_{\{k-1\}}) = H_{\{1\}} * \dots * (Q_{\{k-1\}} - 2 * v_{\{k\}} * (v_{\{k\}}^T * Q_{\{k-1\}}))$ 
Q_flat.assign(N * N, 0.0); // создание единичной матрицы
#pragma omp parallel for num_threads(thread_num)
for (ptrdiff_t i = 0; i < N; ++i) {
    Q_flat[i * N + i] = 1.0;
}

std::vector<T> _2vTQ(N, 0.0);
for (ptrdiff_t k = N - 2; k >= 0; --k)
{
    T* v_k = &all_v[k * N]; // взятие вектора vk
    ptrdiff_t v_length = N - k; // длина вектора vk

    //  $2v^T Q$  параллельно по столбцам
#pragma omp parallel for num_threads(thread_num)
    for (ptrdiff_t j = 0; j < N; ++j) {
        double sum = 0.0;
        for (int i = 0; i < v_length; ++i) {
            ptrdiff_t row = k + i;
            sum += v_k[i] * Q_flat[row * N + j];
        }
        _2vTQ[j] = 2.0 * sum;
    }
}

```

```

// Q - v * 2v^TQ параллельно по строкам
#pragma omp parallel for num_threads(thread_num)
    for (int i = 0; i < v_length; ++i) {
        ptrdiff_t row = k + i;
        double vi = v_k[i];
        T* Q_row = &Q_flat[row * N];

        for (ptrdiff_t j = 0; j < N; ++j) {
            Q_row[j] -= vi * _2vTQ[j];
        }
    }

#pragma omp parallel for num_threads(thread_num) collapse(2)
    for (ptrdiff_t i = 0; i < N; ++i) {
        for (ptrdiff_t j = 0; j < N; ++j) {
            R[i][j] = R_flat[i * N + j];
            Q[i][j] = Q_flat[i * N + j];
        }
    }
};

```

Приложение 3. QR-разложение с использованием отражений Хаусхолдера, реализация in-place алгоритма

```
#pragma once

#include <vector>

#include "../IQRSolver.h"
#include "../Matrix/TransposedMatrix.h"

template <typename T>
class HouseholderWithNormVInplace : public IQRSolver<T> {
public:
    // третья версия, на основе второй. v вычисляется проще для уменьшения роста
    // ошибки, нормированный v сохраняется в R
    virtual void QR_decomposition(
        const std::vector<std::vector<T>>& A,
        std::vector<std::vector<T>>& Q,
        std::vector<std::vector<T>>& R) override
    {
        const size_t N = A.size();
        TransposedMatrix<T> A_T(A), R_T(A);
        TransposedMatrix<T> Q_T(N, N, 0.0);

        // вычисление R
        for (ptrdiff_t k = 0; k < N - 1; ++k) {
            double tau = 0.0, sigma = 0.0;

            // вычисление v, tau, sigma
            // sigma = -sign(Akk) * ||Ak||
            // v = Ak - sigma*(1,0...0)
            // tau = 2 / (vTv)

            double sum = 0.0;
#pragma omp parallel for num_threads(thread_num) reduction(+:sum)
            for (ptrdiff_t i = k; i < N; ++i) {
                sum += R_T.At(i, k) * R_T.At(i, k);
            }
            double norm = sqrt(sum);
            sigma = -sign(R_T.At(k, k)) * norm;

            // вычисляем v, храним без k нулей в начале
            std::vector<T> v(N - k);
            for (ptrdiff_t i = 1; i < N - k; ++i) {
                ptrdiff_t row = k + i;
                v[i] = R_T.At(row, k);
            }
            v[0] = R_T.At(k, k) - sigma;

            double denominator = 0.0;
#pragma omp parallel for num_threads(thread_num) reduction(+:denominator)
            for (ptrdiff_t i = 0; i < N - k; ++i) {
                denominator += v[i] * v[i];
            }

            if (denominator != 0) {
                tau = 2.0 / denominator;
            }

            // сохраняем нормированный v для Q (v[0]=1) в R под диагональю
            if (v[0] != 0) {
                double scale = 1.0 / v[0];
#pragma omp parallel for num_threads(thread_num)
```

```

        for (ptrdiff_t i = 1; i < N - k; ++i) {
            ptrdiff_t row = k + i;
            R_T.At(row, k) = v[i] * scale;
        }
        R_T.At(k, k) = sigma;

        // применяем отражения к правой части
#pragma omp parallel for num_threads(thread_num)
        for (ptrdiff_t col = k + 1; col < N; ++col) {
            double vTR = 0;
            // Вычисляем vTR = vT * столбец col
            for (ptrdiff_t i = 0; i < N - k; ++i) {
                ptrdiff_t row = k + i;
                vTR += v[i] * R_T.At(row, col);
            }

            double scale = tau * vTR;

            // обновляем столбец col: R[:,col] -= v * scale
            for (ptrdiff_t i = 0; i < N - k; ++i) {
                ptrdiff_t row = k + i;
                R_T.At(row, col) -= v[i] * scale;
            }
        }
    }

    // вычисление Q
    for (ptrdiff_t i = 0; i < N; ++i) {
        Q_T.At(i, i) = 1.0;
    }

    // применяем отражения в обратном порядке
    for (ptrdiff_t k = N - 2; k >= 0; --k) {
        // берём вектор v из R
        ptrdiff_t m = N - k; // размер v
        std::vector<T> v(m);
        v[0] = 1.0;

        for (ptrdiff_t i = 1; i < m; ++i) {
            ptrdiff_t row = k + i;
            v[i] = R_T.At(row, k);
            R_T.At(row, k) = 0; // очищаем
        }

        // tau = 2 / (vTv)
        double sum = 1.0; // v[0] = 1
#pragma omp parallel for num_threads(thread_num) reduction(+:sum)
        for (ptrdiff_t i = 1; i < m; ++i) {
            sum += v[i] * v[i];
        }
        double tau = 2.0 / sum;

        // H = I - tau * v * vT к Q[k:N, :]
#pragma omp parallel for num_threads(thread_num)
        for (ptrdiff_t j = 0; j < N; ++j) {
            // vTQ = vT * Q[k:N, col]
            double vTQ = Q_T.At(k, j); // v[0] = 1

            for (ptrdiff_t i = 1; i < m; ++i) {
                ptrdiff_t row = k + i;
                vTQ += v[i] * Q_T.At(row, j);
            }
            vTQ *= tau;

```

```

        // Q[k][j] -= vTQ * v
        Q_T.At(k, j) -= vTQ; // v[0] = 1

        for (ptrdiff_t i = 1; i < m; ++i) {
            ptrdiff_t row = k + i;
            Q_T.At(row, j) -= vTQ * v[i];
        }
    }

    // перевести в обычную матрицу
    Q = Q_T.Transpose();
    R = R_T.Transpose();
}
private:
static double sign(double x)
{
    if (x > 0.0)
        return 1.0;
    if (x < 0.0)
        return -1.0;
    return 1.0;
}
};

```

Приложение 4. QR-разложение с использованием вращений Гивенса, первая реализация

```
#pragma once

#include <vector>
#include <cmath>

#include "../IQRSolver.h"

template <typename T>
class GivensMethodBasic : public IQRSolver<T> {
public:
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
        std::vector<std::vector<T>>& Q,
        std::vector<std::vector<T>>& R) override
    {
        auto N = A.size();
        // матрица Q будет храниться транспонированной
        Q = std::vector<std::vector<T>>(N, std::vector<T>(N, 0));
        R = A;

        for (int i = 0; i < N; ++i)
            Q[i][i] = T(1);

        for (int j = 0; j < N - 1; ++j) {
            for (int i = j + 1; i < N; ++i) {
                // приведение к double для точных проверок и вычислений
                double Rjj = static_cast<double>(R[j][j]);
                double Rij = static_cast<double>(R[i][j]);

                // проверки малости (пороги для double)
                if (std::abs(Rij) < 1e-11)
                    continue;

                double sqrt_val = std::sqrt(Rjj * Rjj + Rij * Rij);
                if (std::abs(sqrt_val) < 1e-11)
                    continue;

                double c = Rjj / sqrt_val;
                double s = -Rij / sqrt_val;

                // вычисление матрицы R путём вращения строк
                #pragma omp parallel for num_threads(thread_num) if (N >= 1000)
                for (int k = j; k < N; ++k) {
                    T temp = static_cast<T>(R[j][k] * c - R[i][k] * s);
                    R[i][k] = static_cast<T>(R[j][k] * s + R[i][k] * c);
                    R[j][k] = temp;
                }

                // сохранение коэффициентов вращения
                double tau = s / (1.0 + c);
                R[i][j] = static_cast<T>(tau);
            }
        }

        for (int j = 0; j < N - 1; ++j)
        {
            for (int i = j + 1; i < N; ++i)
            {
                if (std::abs(R[i][j]) < 1e-11) continue;
                // восстановление коэффициентов вращения
                double tau = static_cast<double>(R[i][j]);
```



```

double tau2 = tau * tau;
double denom = 1.0 + tau2;
double c = (1.0 - tau2) / denom;
double s = 2.0 * tau / denom;

// вычисление матрицы Q_T путём вращения строк
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
for (int k = 0; k < N; ++k)
{
    // обращение к элементам транспонированной матрицы Q
    T temp = static_cast<T>(c * Q[j][k] - s * Q[i][k]);
    Q[i][k] = static_cast<T>(s * Q[j][k] + c * Q[i][k]);
    Q[j][k] = temp;
}
R[i][j] = T(0);
}
// транспонирование
for (int i = 0; i < N; ++i)
{
    for (int j = 0; j < i; ++j)
        std::swap(Q[i][j], Q[j][i]);
}
};

```

Приложение 5. QR-разложение с использованием вращений Гивенса, Q и R в одной структуре данных

```
#pragma once

#include <vector>
#include <cmath>

#include "../IQRSolver.h"
#include "math.h"

template <typename T>
class GivensQRInOneStruct : public IQRSolver<T> {
public:
    // оптимизированная версия. уменьшены обращения к памяти. QR хранятся в одной
    // структуре, каждый элемент попарно рядом в памяти. Q записана по столбцам, R -- по
    // строкам.
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
std::vector<std::vector<T>>& Q, std::vector<std::vector<T>>& R) override
    {
        auto N = A.size();
        QRMatrix<T> qr(A);

        for (auto j = 0; j < N - 1; j++)
        { // зануляем весь столбец j
            for (auto i = j + 1; i < N; i++)
            { // под главной диагональю
                double Rjj = qr.RAt(j, j);
                double Rij = qr.RAt(i, j);

                if (std::abs(Rij) < 1e-11 * std::max(std::abs(Rjj), 1.0)) {
                    continue; // относительные числа
                }

                if (std::abs(Rij) < 1e-11 * std::abs(Rjj)) { // R[i][j]
                    пренебрежимо мал по сравнению с R[j][j]
                    continue;
                }

                if (std::abs(Rij) < 1e-11) { // уже 0
                    continue;
                }

                auto sqrt = std::sqrt(Rjj * Rjj + Rij * Rij);
                if (std::abs(sqrt) < 1e-11) { // для корректировки ошибки
                    игнорируется малый знаменатель
                    continue;
                }
                double c = Rjj / sqrt, s = -Rij / sqrt;

#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
                for (auto k = 0; k < N; k++)
                {
                    if (k >= j)
                    { // меняются только две строки -- i и j
                        auto temp = qr.RAt(j, k) * c - qr.RAt(i, k) * s;
                        qr.RAt(i, k) = qr.RAt(j, k) * s + qr.RAt(i, k) *
c;
                        qr.RAt(j, k) = temp;
                    }

                    // меняются только два столбца -- i и j
                    auto temp = c * qr.QAt(k, j) - s * qr.QAt(k, i);

```

```

        qr.QAt(k, i) = s * qr.QAt(k, j) + c * qr.QAt(k, i);
        qr.QAt(k, j) = temp;
    }
}

R = Q = std::vector<std::vector<T>>(N, std::vector<T>(N));
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
for (int i = 0; i < N; ++i)
{
    for (int j = 0; j < N; ++j)
    {
        R[i][j] = qr.RAt(i, j);
        Q[i][j] = qr.QAt(i, j);
    }
}

private:
template <class T>
class QRMatrix
{
public:
    QRMatrix(const std::vector<std::vector<T>>& A)
        : _data(A.size()* A.size())
        , N(A.size())
    {
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int i = 0; i < N; ++i)
        {
            QAt(i, i) = 1.0;
            for (int j = 0; j < N; ++j)
            {
                RAt(i, j) = A[i][j];
            }
        }

        T& RAt(size_t row, size_t col)
        {
            return _data[row * N + col].r;
        }
        T& QAt(size_t row, size_t col) {
            return _data[col * N + row].q;
        }

private:
    struct QRElement {
        T q;
        T r;
    };
    std::vector<QRElement> _data;
    size_t N;
};
};

```

Приложение 6. QR-разложение с использованием вращений Гивенса, векторизованные вычисления

```
#pragma once

#include <vector>
#include <cmath>
#include <memory>
#include <immintrin.h>
#include <type_traits>

#include "../IQRSolver.h"

template <typename T>
class GivensVectorized : public IQRSolver<T> {
public:
    virtual void QR_decomposition(const std::vector<std::vector<T>>& A,
        std::vector<std::vector<T>>& Q,
        std::vector<std::vector<T>>& R) override
    {
        const int N = A.size();
        if (N == 0) return;

        size_t total = static_cast<size_t>(N) * N;
        auto R_data = std::make_unique<T[]>(total);
        auto Q_data = std::make_unique<T[]>(total);    // Q_T

        // инициализация: R = A, Q = E
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
        for (int i = 0; i < N; ++i) {
            for (int j = 0; j < N; ++j) {
                R_data[i * N + j] = A[i][j];
                Q_data[i * N + j] = (i == j) ? T(1) : T(0);
            }
        }

        // прямой ход: вычисление R
        for (int j = 0; j < N - 1; ++j) {
            for (int i = j + 1; i < N; ++i) {
                const double Rjj = R_data[j * N + j];
                const double Rij = R_data[i * N + j];

                if (std::abs(Rij) < 1e-11)
                    continue;

                const double sqrt_val = std::sqrt(Rjj * Rjj + Rij * Rij);
                if (std::abs(sqrt_val) < 1e-11)
                    continue;

                const double c = Rjj / sqrt_val;
                const double s = -Rij / sqrt_val;

                T* Rj_ptr = &R_data[j * N];
                T* Ri_ptr = &R_data[i * N];

                // векторизованное вращение строк R (ненулевые значения только в
                // столбцах правее j)
                if constexpr (std::is_same_v<T, double>) {
                    size_t k = j;
                    for (; k + 3 < N; k += 4) {
                        __m256d Rj = _mm256_loadu_pd(&Rj_ptr[k]);
                        __m256d Ri = _mm256_loadu_pd(&Ri_ptr[k]);
                    }
                }
            }
        }
    }
};
```

```

        __m256d c_vec = _mm256_set1_pd(c);
        __m256d s_vec = _mm256_set1_pd(s);

        __m256d new_Rj = _mm256_fmsub_pd(Rj, c_vec, _mm256_mul_pd(Ri,
s_vec));
        __m256d new_Ri = _mm256_fmadd_pd(Rj, s_vec, _mm256_mul_pd(Ri,
c_vec));

        _mm256_storeu_pd(&Rj_ptr[k], new_Rj);
        _mm256_storeu_pd(&Ri_ptr[k], new_Ri);
    }
    for (; k < N; ++k) {
        T Rj = Rj_ptr[k], Ri = Ri_ptr[k];
        Rj_ptr[k] = Rj * c - Ri * s;
        Ri_ptr[k] = Rj * s + Ri * c;
    }
}
else if constexpr (std::is_same_v<T, float>) {
    size_t k = j;
    for (; k + 7 < N; k += 8) {
        __m256 Rj = _mm256_loadu_ps(&Rj_ptr[k]);
        __m256 Ri = _mm256_loadu_ps(&Ri_ptr[k]);

        __m256 c_vec = _mm256_set1_ps(c);
        __m256 s_vec = _mm256_set1_ps(s);

        __m256 new_Rj = _mm256_fmsub_ps(Rj, c_vec, _mm256_mul_ps(Ri,
s_vec));
        __m256 new_Ri = _mm256_fmadd_ps(Rj, s_vec, _mm256_mul_ps(Ri,
c_vec));

        _mm256_storeu_ps(&Rj_ptr[k], new_Rj);
        _mm256_storeu_ps(&Ri_ptr[k], new_Ri);
    }
    for (; k < N; ++k) {
        T Rj = Rj_ptr[k], Ri = Ri_ptr[k];
        Rj_ptr[k] = Rj * c - Ri * s;
        Ri_ptr[k] = Rj * s + Ri * c;
    }
}

// сохраняем c и s в виде tau
double tau = s / (1.0 + c);
R_data[i * N + j] = static_cast<T>(tau);
    }
}

// обратный ход: построение Q_T по сохранённым tau
for (int j = 0; j < N - 1; ++j) {
    for (int i = j + 1; i < N; ++i) {
        if (std::abs(R_data[i * N + j]) < 1e-11) continue;

        double tau = static_cast<double>(R_data[i * N + j]);
        double tau2 = tau * tau;
        double denom = 1.0 + tau2;
        double c = (1.0 - tau2) / denom;
        double s = 2.0 * tau / denom;    // восстанавливаем c и s

        T* Qj_ptr = &Q_data[j * N];
        T* Qi_ptr = &Q_data[i * N];

        // применяем вращение строк Q_T
        if constexpr (std::is_same_v<T, double>) {
            size_t k = 0;

```

```

        for (; k + 3 < N; k += 4) {
            __m256d Qj = _mm256_loadu_pd(&Qj_ptr[k]);
            __m256d Qi = _mm256_loadu_pd(&Qi_ptr[k]);

            __m256d c_vec = _mm256_set1_pd(c);
            __m256d s_vec = _mm256_set1_pd(s);

            __m256d new_Qj = _mm256_fmsub_pd(Qj, c_vec, _mm256_mul_pd(Qi,
s_vec));
            __m256d new_Qi = _mm256_fmadd_pd(Qj, s_vec, _mm256_mul_pd(Qi,
c_vec));

            _mm256_storeu_pd(&Qj_ptr[k], new_Qj);
            _mm256_storeu_pd(&Qi_ptr[k], new_Qi);
        }
        for (; k < N; ++k) {
            T Qj = Qj_ptr[k], Qi = Qi_ptr[k];
            Qj_ptr[k] = Qj * c - Qi * s;
            Qi_ptr[k] = Qj * s + Qi * c;
        }
    }
    else if constexpr (std::is_same_v<T, float>) {
        size_t k = 0;
        for (; k + 7 < N; k += 8) {
            __m256 Qj = _mm256_loadu_ps(&Qj_ptr[k]);
            __m256 Qi = _mm256_loadu_ps(&Qi_ptr[k]);

            __m256 c_vec = _mm256_set1_ps(c);
            __m256 s_vec = _mm256_set1_ps(s);

            __m256 new_Qj = _mm256_fmsub_ps(Qj, c_vec, _mm256_mul_ps(Qi,
s_vec));
            __m256 new_Qi = _mm256_fmadd_ps(Qj, s_vec, _mm256_mul_ps(Qi,
c_vec));

            _mm256_storeu_ps(&Qj_ptr[k], new_Qj);
            _mm256_storeu_ps(&Qi_ptr[k], new_Qi);
        }
        for (; k < N; ++k) {
            T Qj = Qj_ptr[k], Qi = Qi_ptr[k];
            Qj_ptr[k] = Qj * c - Qi * s;
            Qi_ptr[k] = Qj * s + Qi * c;
        }
    }
    R_data[i * N + j] = T(0); // восстанавливаем чистый ноль
}
}

// копирование в выходные матрицы
Q.assign(N, std::vector<T>(N));
R.assign(N, std::vector<T>(N));
#pragma omp parallel for num_threads(thread_num) if (N >= 1000)
for (int i = 0; i < N; ++i) {
    for (int j = 0; j < N; ++j) {
        R[i][j] = R_data[i * N + j];
        Q[i][j] = Q_data[j * N + i]; // транспонирование
    }
}
}
};

```